

Final report for MSc Project

Multi-Class Neuronal Spike Classification with Deep Learning: A Computational Study of Spike-Based and Image-Based Spike Classification Methods

Author name: Mahima Ghosh

CID: 02451677

Supervisor(s): Dr. Huai- Ti Lin

Bence A. Barta (PhD)

Submitted in partial fulfilment of the requirements for the award of MSc in
Human and Biological Robotics from Imperial College London

Abstract

The field of neuroscience has witnessed a growing demand for efficient methods to analyse neural activity, particularly in neuronal spike sorting, which involves identifying and separating individual neuronal signals from multi-unit recordings. Neuronal spike sorting is crucial for decoding neural activity, but traditional methods—relying on manual feature extraction and clustering algorithms—are limited by labour-intensiveness, subjectivity, and difficulty in handling large-scale, noisy datasets. This study addresses these limitations by introducing a novel multi-class spike classification approach using transfer learning with the EfficientNet-B3 model. The methodology transforms raw spike waveforms into grayscale images, allowing the application of deep learning techniques. The EfficientNet-B3 model, fine-tuned with the Convolutional Block Attention Module (CBAM), was trained on ~1.45 million spike images to classify spikes into neuron classes, focusing on 1,000 neuron labels.

The results demonstrate a significant improvement in classification accuracy, with the Image-based Transfer Learning Spike Classifier (ITSC) achieving an accuracy of 86.35%, outperforming both the Spike-based Classifier (SC) and the Image-based Spike Classifier Non-pretrained (ISC), which achieved accuracies of 78.3% and 79.75%, respectively. This approach not only surpasses traditional spike sorting methods but also highlights the efficacy of using deep learning models in neural data analysis, specifically for large-scale, multi-class classification tasks. The study's findings suggest that integrating deep learning with transfer learning offers a scalable, efficient solution for neuronal spike sorting, with potential applications across various domains.

Keywords

Neuroscience, Neural Activity Analysis, Spike Sorting, Multi-Class Classification, Deep Learning, Transfer Learning, Attention Mechanism.

Acknowledgements

I extend my heartfelt thanks to Dr. Huai-Ti Lin for his invaluable supervision throughout this project.

I am deeply grateful to Mr. Bence Barta (PhD), my main point of contact, for his exceptional guidance, steadfast support, and invaluable advice. His dedication and expertise have been fundamental to the success of this research.

I also wish to express my profound gratitude to my best friend, Sridhar Sola (Robot Engineer, Dyson), and my parents for their constant encouragement, understanding, and support during this journey. Their belief in me has been a source of immense motivation and strength.

Special thanks to Mr. Bence Barta (PhD) and Charlie Hadfield for their assistance during technical challenges. Without their help, this project would not have been a success.

Contents

Abstract	2
Keywords	2
Acknowledgements	3
1. Introduction	5
2. Methods.....	8
2.1. Spike Sorting vs Spike Classification	8
2.2. Dataset and Label	8
2.3. Class and Model Selection Criteria	8
2.4. Spike-based Classifier (SC)	9
2.5. Image-based Spike Classifier (ISC).....	12
2.6. Image-based Transfer Learning Spike Classifier (ITSC)	13
2.6.1. Conversion of Spike Waveforms to Images	13
2.6.2. Pretrained Model Selection	15
2.6.3. Fine-Tuning the Model.....	17
2.6.4. Training and Optimization	18
3. Results	19
3.1. SC Model Results.....	19
3.2. ITSC Model Results	21
3.3. ISC Model Results	24
3.4. Final Comparison	25
4. Discussion	27
5. References	29
Annex.....	33
Transfer Learning	33
Class Selection criteria	35
Model Selection.....	36
EfficientNet-B3	37
CBAM.....	39
Model Fine-tuning	41
Model Optimization	41
Model Development Code	43

1. Introduction

The analysis of neural activity is crucial for understanding brain function and behaviour. A key technique in this field is spike sorting, which involves identifying and separating individual neuronal signals from multi-unit extracellular recordings. Accurate spike sorting is vital for decoding neural activity at the single-neuron level. Traditional methods, such as Principal Component Analysis (PCA) followed by clustering algorithms like K-means or Gaussian Mixture Models (GMM), are commonly used but have significant limitations. These methods are labour-intensive, time-consuming, and prone to subjective biases, particularly when handling complex datasets with high variability or noise [1] (Refer Table 1: Existing Spike Sorting Methodologies).

Recent advancements in machine learning, particularly deep learning, have begun to address these limitations. Convolutional Neural Networks (CNNs) have shown the ability to learn hierarchical feature representations directly from raw data, eliminating the need for manual feature extraction [2]. Transfer learning, which involves adapting a model pre-trained on a large dataset to a specific task with less data, has also emerged as a powerful tool. This approach has proven successful in various domains by enhancing classification accuracy and efficiency while reducing the need for large, labelled datasets.

Related Work

Traditional spike sorting methods rely heavily on manual feature extraction followed by clustering algorithms. While effective in some cases, these methods are limited by their dependence on manual intervention, assumptions about data distribution, and sensitivity to noise. They often struggle with highly variable or noisy datasets, leading to reduced accuracy and scalability [3] [1].

Deep learning offers a solution to these challenges by automating feature extraction and learning from raw data. Shen et al., 2019 [4] demonstrated that CNNs could outperform traditional methods by directly learning from raw spike waveforms. This capability allows CNNs to capture complex patterns in neural data that are often missed by manual feature extraction methods. Recent studies have expanded on this by exploring various deep learning architectures, such as Long Short-Term Memory (LSTM) networks and autoencoders, which are designed to handle the temporal dynamics of spike trains more effectively. LSTMs have been shown to model temporal dependencies in spike data, improving the accuracy of spike classification [5]. Autoencoders, on the other hand, have been used to reduce the dimensionality of spike data while preserving critical features, further enhancing the efficiency of the spike sorting process [6]. However, training deep neural networks from scratch requires large amounts of labelled data and significant computational resources, which are often unavailable in neuroscience [7].

Transfer learning addresses these issues by adapting pre-trained models to specific tasks with less data. This approach leverages the knowledge gained from the initial task, such as image classification, to improve performance in spike sorting (Refer Annex: Transfer

Learning). Studies by Yamins et al. in 2014 [8] and Maida et.al in 2019 [9] have shown that transfer learning can enhance classification accuracy and efficiency in neural data analysis, outperforming models trained from scratch.

Table 1: Existing Spike Sorting Methodologies

Method	Feature Extraction	Classification	Accuracy	Limitations
PCA	Manual	K-means, GMM	~70% [1]	Loss of information, manual intervention
Wavelet Transform	Manual	SVM, Clustering	75-80% [10]	Extensive manual tuning, sensitive to parameters
Template Matching	Template-based	Matching	70-75% [11]	Variability in spike shapes, noise sensitivity
Clustering Algorithms	Manual	K-means, GMM	70-80% [3] [12]	Quality of features, noise sensitivity
SVM	Manual	SVM	80-85% [3]	Dependent on features and kernel functions
HMM	Manual	HMM	~80% [13]	Computationally intensive
GMM	Manual	EM technique	~80% [14]	computationally more intensive and requires a good initialization to avoid convergence to local optima
MultiFq	Automated	PCA-Km	Increase by 35.04%	Extensive optimization of the filtering process
Data Subdivision and Unification	Manual	subdividing large electrophysiological datasets into smaller, more manageable segments, followed by unification of the sorted results	Accuracy increase by 56.87% while computational time is reduced up to 84.7% [15]	Computationally expensive
CNN	Automated	CNN	85-90% [1]	Requires large, annotated datasets
Autoencoders	Automated	Autoencoder	80-85% [16]	Reconstruction errors, tuning required

Research Gap

Despite these advancements, several key challenges remain. Most deep learning models for spike sorting are still trained from scratch, requiring extensive computational resources. This approach is not scalable or practical in many research environments. Additionally, the focus has primarily been on binary or small-scale multi-class classification, which does not reflect the complexity of real-world neural data that often requires large-scale, multi-class classification.

There is also limited research on using advanced architectures like EfficientNet, which optimizes performance through compound scaling, or integrating attention mechanisms like CBAM, which can enhance feature extraction. The potential of transfer learning to reduce training time and improve model generalization, particularly for noisy and variable datasets, remains underexplored in spike sorting.

Objectives

This project aims to address these gaps by exploring the application of deep learning techniques for **multi-class spike classification**. The specific objectives of this study are to develop:

- a) Customize ResNet-18 for spike waveform classification: Adapt a deep neural network to directly handle raw spike data and evaluate its effectiveness.
- b) Evaluate an EfficientNet-B3 model trained from scratch on spike images: Use this as a baseline comparison against transfer learning methods
- c) Fine-tune a pre-trained EfficientNet-B3 for image-based spike classification: Convert spike waveforms to images and enhance accuracy with minimal training time.
- d) Compare all approaches and identify improvements: Systematically assess model performance to find the best approach for multi-class spike sorting.

Hypotheses

To achieve the above objective, the following hypothesis are tested:

(H1) *Image Processing Hypothesis*: Leveraging image processing techniques enhances spike classification accuracy.

(H2) *Deep Learning Spike Classification*: Deep learning, particularly transfer learning, improves spike classification performance, faster training and higher accuracy.

By addressing the limitations of traditional methods and exploring novel approaches, this study aims to set a new benchmark for neuronal spike classification, contributing to a more comprehensive understanding of neural activity. The findings from this study have the potential to enhance the accuracy and efficiency of spike classification, ultimately facilitating more detailed analyses of neuronal behaviour and brain function.

2. Methods

The proposed methodology follows 3 novel but simple approaches to multi-class spike classification as shown below. To find the code for all the models developed, refer Annex: Model Development Code.

Table 2: Proposed New Spike Sorting Methods

Method	Feature Extraction	Classification	Limitations
SC Model	Automated using custom parameters	ResNet-18	Adaptation for time series data
ISC Model	Automated using non-pretrained parameters	Non-pretrained EfficientNet-B3 with CBAM	Requires large training hours
ITSC Model	Automated using pretrained parameters	Pretrained EfficientNet-B3 with CBAM	Fine-tuning required

2.1. Spike Sorting vs Spike Classification

Spike sorting aims to group neuronal spikes from a single recording channel based on shared characteristics (e.g., waveform shape). It involves feature extraction and clustering algorithms [1]. In contrast, spike classification assigns spikes to predefined classes using supervised learning. Classification explicitly categorizes spikes into known neuron types, while sorting focuses on similarity-based grouping [3].

The goal here is to classify spikes into 1,000 distinct classes, each representing a specific neuron. This explicit categorization differentiates the task from spike sorting, which does not use predefined classes but rather identifies clusters of similar spikes.

2.2. Dataset and Label

The dataset¹ used in this study comprises neural data recorded from dragonflies, providing a rich source of spike waveforms for analysis. The dataset includes 31 separate recordings, capturing spikes across each neuron firing. In total, the dataset contains 1,435,932 spikes (~1.45 million), each represented as a spike waveform with a length of 81 time-steps, recorded at 60,000 Hz across 3544 distinct neurons². The labelled dataset acts as the ground truth for this entire project.

2.3. Class and Model Selection Criteria

Initially, the goal was to classify neuronal spikes into 3544 distinct classes using a dataset of 1.45 million samples. However, due to technical and computational challenges, the focus shifted to a subset of 1000 classes, resulting in 443K samples, a reduction of approximately

¹ The dataset has been compiled by Dr. Daniel Ko and Bence Barta (PhD), Department of Bioengineering, Imperial College London.

² In total, the dataset contains ~1.45 million spikes and corresponding ~1.45 million labels that range from the values of 0-3544. For this study, the spikes and their corresponding labels used are 0-999.

69.2% in the number of samples and 71.8% in the number of classes (Refer Annex: Class Selection criteria).

In this project, two approaches were used: a Spike-based classification with ResNet-18 for raw spike recordings, and an Image-based classification using EfficientNet-B3 with CBAM for superior performance in image-based classification. This combination ensured effective spike classification. This combination ensured a comprehensive and effective approach to classifying neuronal spikes into distinct classes (Refer Annex: Model Selection).

The analysis conducted in this project is crucial for advancing further studies. By achieving faster and more accurate spike classification, the focus can be on applying these classified spikes to understand neural activity better and explore real-world applications. This efficiency will significantly contribute to neuroscience research and practical implementations. Hence the focus is on faster training and achieving higher accuracy.

2.4. Spike-based Classifier (SC)

The SC approach uses a ResNet-inspired architecture, which is traditionally used for image classification but has been adapted here to handle one-dimensional time-series data. The SC aims to classify spikes into distinct neuronal classes without the need for converting the waveforms into images, thus preserving the temporal characteristics of the spike signals.

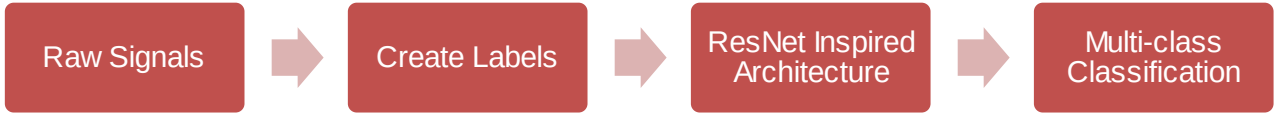


Figure 1: Workflow of the Spike-Based Classifier The depicted workflow outlines the methodology employed for spike-based classification. Notably, the “create labels” step signifies that the provided dataset was pre-labelled, with each folder corresponding to a specific neuron number. To operationalize this, a Python script was developed to extract labels from the folder names. Subsequently, the Spike-Based Classifier (SC) model was implemented. Raw neural signals, accompanied by their corresponding labels, constituted the training data for a ResNet-Inspired model. The outcome achieved was successful multi-class classification.

ResNet Inspired Model and Adaptation

ResNet (Residual Network) is known for its deep learning capabilities, particularly its ability to train very deep networks without falling into the vanishing gradient problem by implementing residual connections [17]. The input layer of the spike sorting convolutional network (SC) processes a one-dimensional vector representing a spike waveform, consisting of 81 time steps. This input shape is maintained throughout the network to ensure that the temporal dynamics of the spikes are fully captured, avoiding potential information loss that might occur during transformation into other data formats, such as images.

$$x = [x_1, x_2, \dots, x_n]$$

1

where the input to the SC model is a 1D vector representing a spike waveform. Here $n=81$ as each spike waveform consist of 81 time steps.

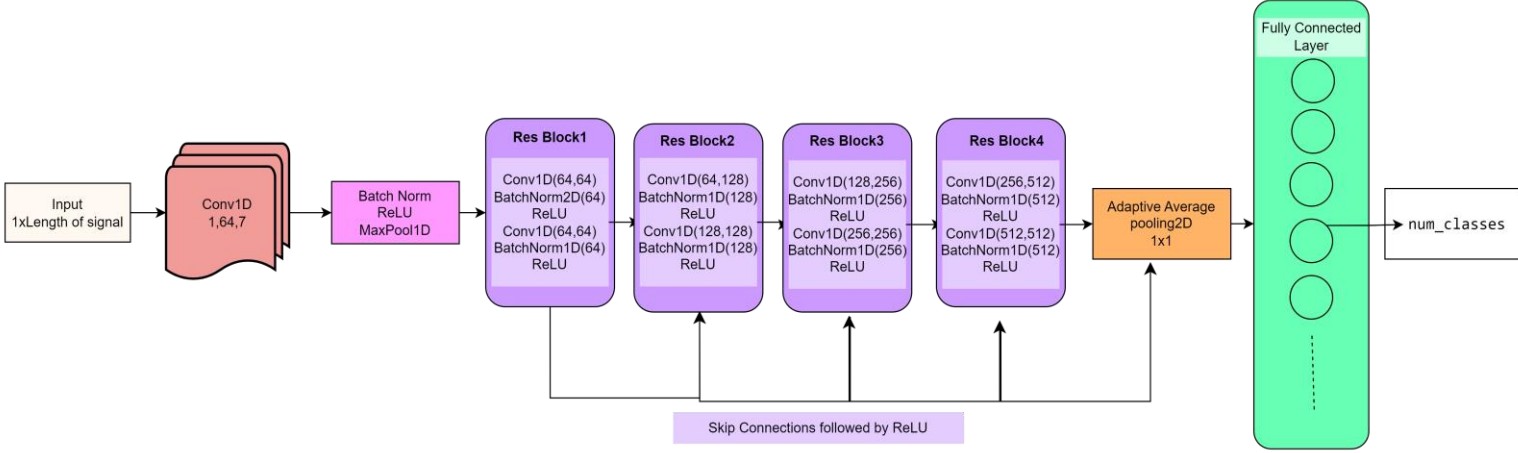


Figure 2: ResNet-Inspired Architecture for time-series data (spike waveforms). In this study, the ResNet-18 architecture is modified to process 1D spike waveforms—a departure from its original 2D image context. The critical innovation lies in the incorporation of skip connections, which mitigate the vanishing gradient challenge. These connections enable deeper network architectures, essential for capturing intricate neuronal features. The resulting model achieves accurate and efficient classification of distinct neuronal sources.

In the 1D convolutional layers, the standard 2D convolutional layers of ResNet are replaced with 1D convolutional layers. These 1D convolutions slide over the time axis of the spike waveforms, detecting temporal patterns within the data. A small filter size of 3 time steps was chosen to capture fine-grained temporal features of the spike waveform, while a stride of 1 was used to maintain temporal resolution. This approach is ideal for capturing local dependencies in time-series data, such as the sequential voltage changes in spike waveforms, allowing the model to learn intricate patterns critical for distinguishing between spikes from different neurons. The

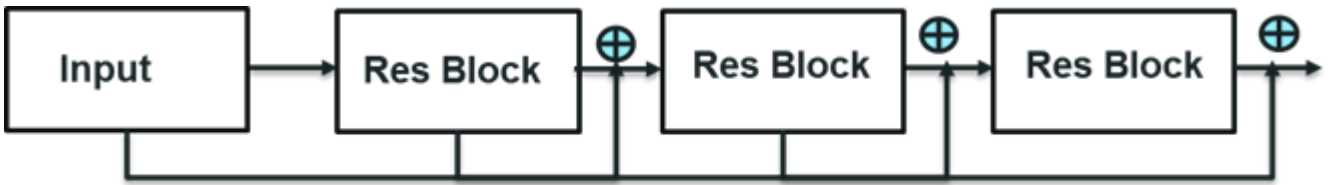


Figure 3: Residual Blocks with skip connections. The $\oplus$ symbolizes ReLU activation implemented after each of the residual blocks. The skip connections ensure efficient learning of the model. This figure illustrates the architecture of residual blocks used in the ResNet-inspired model. The skip connections, represented by arrows bypassing one or more layers, play a crucial role in mitigating the vanishing gradient problem. By allowing the gradient to flow directly through these connections, the network can maintain effective learning even as it becomes deeper. This mechanism ensures that the model can learn both the identity function and any additional transformations necessary for accurate classification, thereby enhancing training stability and performance.

1D convolutional layers slide over the time axis of the spike waveforms, detecting temporal patterns within the data. The convolution operation for a given layer can be expressed as:

$$y_i = \sigma\left(\sum_{j=1}^k w_i \cdot x_{i+j-1} + b\right) \quad 2$$

where, $w_i \cdot x_{i+j-1}$ is the filter, k is the filter size, b is the bias, σ is the activation function (ReLU in this case) [18].

Residual connections, like those in the original ResNet architecture, were introduced between layers to create residual blocks.

$$y = x + F(x, \{W_i\}) \quad 3$$

where, $F(x, \{W_i\})$ represents the residual function [19]. Residual connections enable the network to learn both the identity function, passing inputs unchanged, and any additional transformations necessary for classification. This is particularly important in deep networks, where it helps mitigate issues with gradient flow and training stability (Figure 3).

Rectified Linear Unit (ReLU) activations were used after each convolutional layer to introduce non-linearity to the system followed by max pooling that focuses on reducing the dimensions. After the convolutional and pooling layers, the output feature map is flattened and passed through fully connected (dense) layers. The final dense layer has an output dimension equal to the number of neuron classes (1000), with a SoftMax activation function to produce class probabilities.

Training and Optimization

The dataset was randomly divided into training (80%) and testing (20%) subsets to evaluate the model's performance. The cross-entropy loss function was used to measure the difference between the predicted class probabilities and the true class labels. Cross-entropy is widely used in classification tasks, especially in deep learning, because it penalizes incorrect predictions more heavily, thus driving the model to improve its accuracy.

$$L = -\sum_{i=1}^N y_i \log(\hat{y}_i) \quad 4$$

where, y_i is the true label and $\hat{y}_i$ is the predicted probability for the i^{th} class [20]. The Adam optimizer was selected for training the SC model with a fixed learning rate of 0.0001 for this specific task. This decision was made after experimenting with learning rates ranging from 0.1 to 0.00001, where it was observed that the model only effectively learned at the 0.0001 learning rate. The update rules for the parameters θ was done as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} L_t \quad 5$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} L_t)^2 \quad 6$$

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad 7$$

$$\widehat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad 8$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t + \epsilon}} \quad 9$$

Where m_t and v_t are the first and second moment estimates, α is the learning rate, and ϵ is a small constant [21]. Adam combines the advantages of two other popular optimizers: *AdaGrad* and *RMSProp*, allowing it to handle sparse gradients on noisy data effectively [22]. Dropout layers were introduced after dense layers with a dropout rate of 0.5. Dropout randomly sets a fraction of input units to zero at each update during training, preventing the model from overfitting. Regularization techniques like dropout help in improving the generalization ability of the model, ensuring that it does not simply memorize the training data but learns to generalize to new, unseen data.

In conclusion, the SC model employs 1D convolutions to detect both local and global temporal patterns within spike waveforms. Its ResNet-inspired architecture, featuring residual connections, addresses the vanishing gradient problem, allowing for deeper networks. As a result, the SC model excels at capturing intricate details of neuronal spikes and represents a new approach in spike classification.

2.5. Image-based Spike Classifier (ISC)

The ISC leverages the predefined EfficientNet-B3 architecture (Refer 2.6.2. Pretrained Model Selection for model selection criteria) without using the pretrained weights, rather trains from scratch to perform multi-class classification. This approach allows the model to learn features specific to neuronal spike classification directly from the data, without being influenced by prior knowledge from other domains (such as ImageNet). While this requires more computational resources and training time, it offers the advantage of developing a model that is highly specialized for the task at hand.

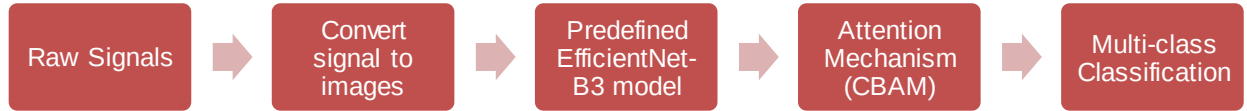


Figure 4: Workflow of the Image-based Spike Classifier (ISC). The spike images used here are the same ones used for the ITSC model. These images are then used to train the predefined EfficientNet model and then used for multi-class classification. Refer 2.6.1. Conversion of Spike Waveforms to Images

The ISC initializes the EfficientNet-B3 architecture with random weights. This means that all convolutional filters, biases, and fully connected layers start without any pre-learned knowledge, requiring the model to learn all features directly from the spike images. This is typically done using methods such as Xavier or He initialization, which are designed to maintain the variance of the weights across layers, preventing issues with vanishing or exploding gradients [23] [24]. The ISC retains the predefined Efficientnet-B3 architecture (Refer Annex:EfficientNet-B3). The ISC model is trained using the exact process as the ITSC (Refer 2.6. Image-based Transfer Learning Spike Classifier (ITSC) for implementation details).

2.6. Image-based Transfer Learning Spike Classifier (ITSC)

The ITSC represents a novel approach to spike sorting and classification by leveraging transfer learning from a pretrained EfficientNet-B3 model, originally developed by Google Research for image classification tasks [25] [26]. This method diverges from traditional spike sorting techniques by converting raw spike waveforms into grayscale images and utilizing the deep feature extraction capabilities of EfficientNet-B3 to classify these images into distinct neuronal classes.

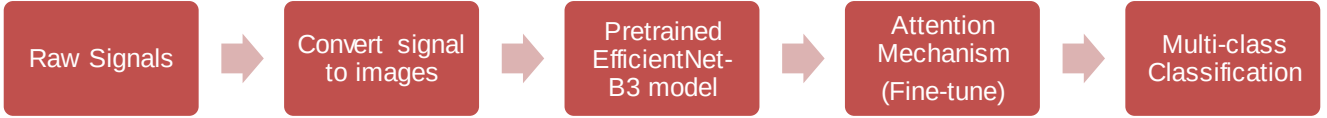


Figure 5: Workflow of the Image-based Transfer Learning Spike Classifier (ITSC). The raw signals, along with the labels were converted to images (Refer 2.6.1. Conversion of Spike Waveforms to Images). Each image is of 224x224 pixels and is labelled and stored in a hdf5 file. These images are then used to train the pretrained EfficientNet-B3 and then fine-tuned with the spike waveforms using CBAM attention module to capture the subtle differences in spike shapes.

Converting spikes to images allows CNNs to exploit their strengths in spatial feature extraction. CNNs are particularly effective at detecting edges, textures, and complex patterns within images, which can correspond to subtle differences in spike waveforms that are crucial for classification [18]. Each pixel in the image encodes information about both the temporal progression and amplitude of the spike, capturing nuances that might be lost in a lower-dimensional representation. Increasing the dimensionality allows the model to capture more complex patterns and interactions between features. The convolutional filters in CNNs can detect local correlations and spatial hierarchies, which are essential for distinguishing between similar but distinct spike waveforms.

2.6.1. Conversion of Spike Waveforms to Images

The input data consists of time-series waveforms representing neuronal spikes. Each waveform contains 81-time steps, providing a detailed temporal profile of the spike waveform. The proposed idea is to leverage image processing methods for signal processing tasks (Refer Hypothesis (H1) in page 5). First the 1D spikes are converted into 2D images that will be the input to the CNN. To leverage the capabilities of a pretrained EfficientNet model, the raw spike waveforms are transformed into 224×224 pixel images. Each spike waveform is plotted as a single row of pixel intensities, effectively capturing the temporal dynamics of the waveform in a visual format [19].

$$I(x, y) = \begin{cases} A(t) & , \text{if } y = f(t) \\ 0 & , \text{otherwise} \end{cases} \quad 10$$

where, $I(x,y)$ is the pixel intensity at the coordinates (x,y) , $A(t)$ is the amplitude of the spike waveform at time (t) and $f(t)$ maps the time step (t) to the corresponding row in the image.

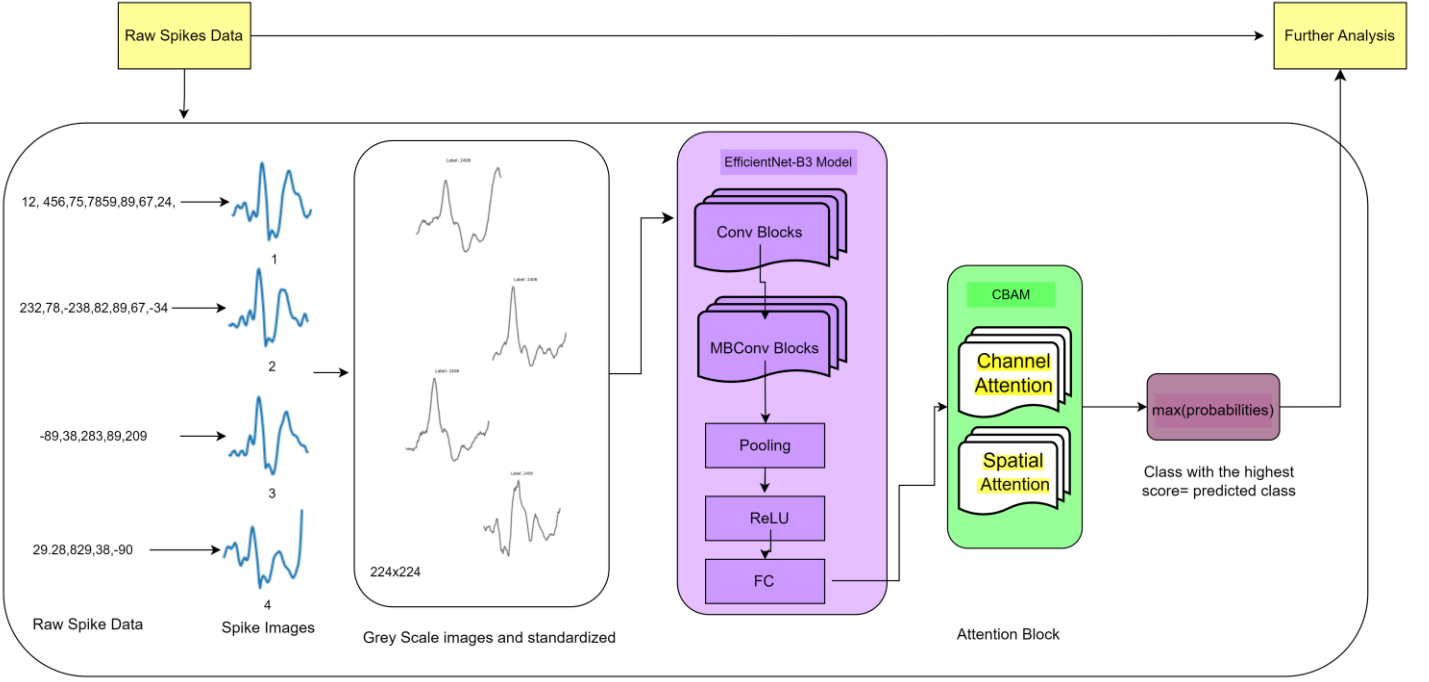


Figure 6: The ITSC model architecture- Here, the raw spikes obtained from the dragonfly neuronal recordings are converted to images and then the pretrained EfficientNet-B3 is fine-tuned on the spike waveforms after extracting low level features using the pretrained weights. The attention block focuses on capturing subtle differences in spike images that could have been missed by the EfficientNet. The CBAM captures the subtle differences that differentiates each spike label from another effectively. The class with the highest score calculated using SoftMax, is the predicted class. The predicted label along with the spike waveforms can then be used for further analysis.

The spike waveforms are represented as (1), and the waveform is then converted to images:

$$I = I(image) = \begin{pmatrix} x_1 & \cdots & x_{32} \\ \vdots & \ddots & \vdots \\ x_{49} & \cdots & x_{81} \end{pmatrix} \quad 11$$

Each I is then reshaped to a 224×224 image, aligning with the input requirements of EfficientNet model. The images generated from spike signals are initially in colour (RGB). However, for training with a pre-trained EfficientNet model, the images are converted to grayscale as colour information is not necessary. The images stored in the directory are loaded batch by batch. This batch processing is crucial to prevent memory overflow, especially given the large number of images. In this case, 1,435,936 (~1.45 million) total images and neuron labels/classes. The standard grayscale conversion is:

$$I_{gray} = 0.2989 I_R + 0.5870 I_G + 0.1140 I_B$$

12

Instead of the standard grayscale conversion as shown in (12), the pixel values are normalized to have a mean of 0.5 and a standard deviation of 0.5 which ensures that the pixel values are centred around zero, which can help in reducing the memory footprint during training. Normalization helps get data within a range and reduces the skewness which helps the model to learn faster and better. After processing each batch, garbage collection is triggered to free up memory, ensuring that the system doesn't run out of resources during the processing of large datasets.

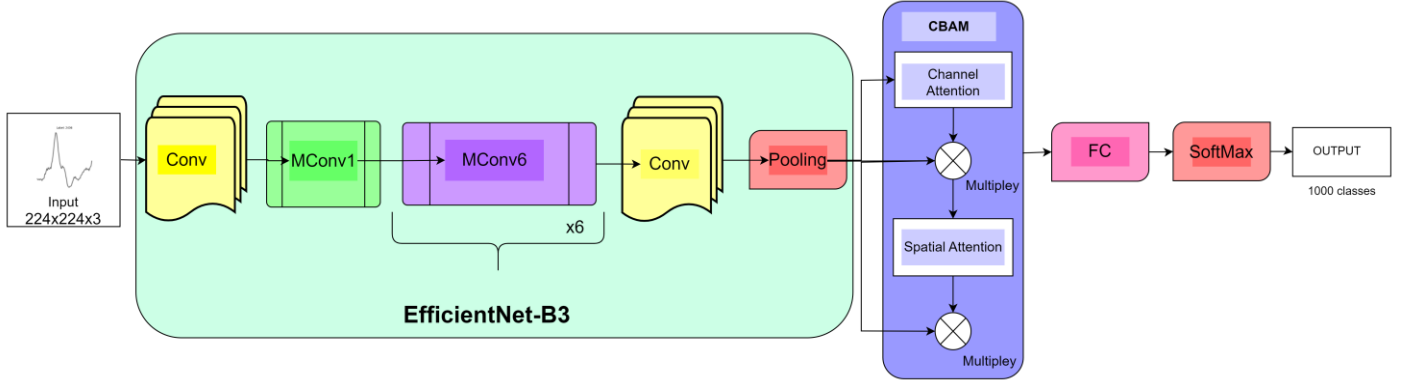


Figure 7: The ITSC model architecture using transfer learning EfficientNet-B3 with CBAM attention mechanism. The Conv is the initial convolution layer to extract basic features. MBConv1 Block is the first mobile inverted bottleneck convolution block. Then there is a series of MBConv6 Blocks, that is the mobile inverted bottleneck convolution blocks with a depth multiplier of 6. Another convolution layer to further refine features. Followed by global average pooling to reduce spatial dimensions. Then the Channel Attention (CBAM) module to focus on important channels. Followed by the Spatial Attention (CBAM) module to focus on important spatial regions. Finally, the Fully Connected (FC) Layer for classification. Then the SoftMax layer to produce class probabilities and the final classification output

2.6.2. Pretrained Model Selection

The model selected for the ITSC is the EfficientNet-B3 with CBAM attention mechanism (Refer Annex: EfficientNet-B3 and, Table 3: Pretrained Model Selection). EfficientNet is a well-established convolutional neural network (CNN) architecture designed for image classification tasks in 2019 [27]. The EfficientNetB3 offers several advantages over ResNet-18. Firstly, the compound scaling method used in EfficientNet allows for a more balanced and efficient scaling of the network, leading to better performance with fewer parameters. Secondly, the use of MBConv blocks with SE optimization enhances the model's ability to capture complex patterns while maintaining computational efficiency. Lastly, EfficientNetB3 has been shown to achieve higher accuracy on various image classification benchmarks compared to ResNet-18, making it a more suitable choice for tasks requiring high precision and efficiency [27].

Table 3: Pretrained Model Selection

Feature	ResNet-18	ResNet-50	EfficientNet-B3
Size	~11.7 million	~25.6 million	~12 million
Computational Resources	~1.8 billion	~4.1 billion	~1.8 billion
Training Epochs (100)	Trains fast but not suitable for this project	Moderate but too complex	Moderate due to compound scaling but more accurate
Accuracy	67.12%	70.34%	86.35%
Training Time (Hours)	15 hours	25 hours	10 hours
Fine-Tuning	High (150 epochs)	High (150-180 epochs)	Low (50 epochs)
Scalability	Limited	Better	High

Table 4: Model Selection Criteria

Model	Accuracy (1000 classes)	Number of Epochs	Training Time (Hours)	Description
ResNet-18	67.12%	100	15	A standard model for comparison, demonstrating good performance with lower computational requirements.
ResNet-50	70.34%	100	25	Deeper architecture provides better accuracy but at the cost of increased training time and resources.
EfficientNet-B3 with CBAM	86.35%	50	10	Combines efficiency and performance, making it ideal for image-based tasks with high accuracy.

The architecture of EfficientNetB3 includes a series of mobile inverted bottleneck convolution (MBConv) blocks, which are enhanced with squeeze-and-excitation (SE) optimization (Refer Annex: EfficientNet-B3). These blocks are designed to improve the model's ability to capture complex patterns in the data while maintaining computational efficiency. The presence of skip connections within these blocks helps mitigate the vanishing gradient problem, which is a common issue in deeper neural networks. EfficientNetB3 has been extensively validated and optimized for 224×224 pixel images, making it suitable for a wide range of image classification tasks (fig 8).

The dataset is analysed for class imbalance. Since some labels may be underrepresented, class weights are computed and applied during training to ensure the model doesn't bias towards more frequent labels. In this case, there was an extreme case of class imbalance that was dealt

by using class weights [28]. In this scenario, underrepresented classes are assigned higher weights while training to avoid bias during training.

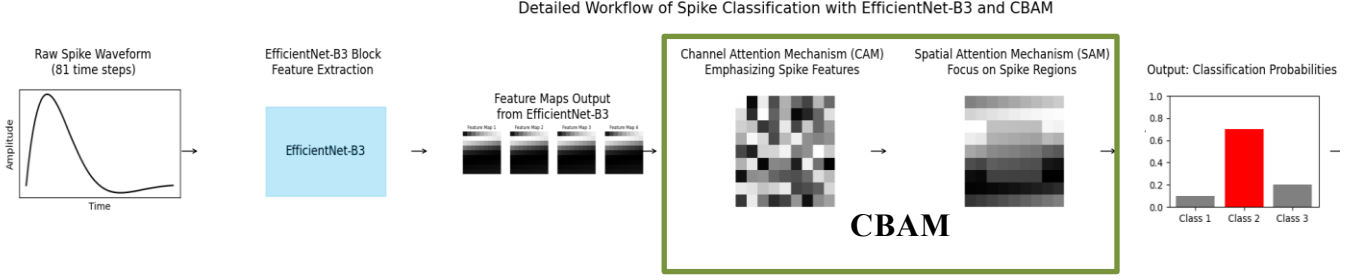


Figure 8: The classification of the spike images is done as follows: the spike image is the input to the EfficientNet-B3, the output after initial convolutions are the feature maps highlighting the spike waveforms feature components. Next an attention mechanism is incorporated in the EfficientNet model that focuses on the channels and spatial information of the spike waveforms. The output of the CBAM then goes through an FC layer, followed by SoftMax to produce the class probabilities that assign the spike image a distinct class/neuron label. Refer Annex: CBAM for computational details.

The ITSC leverages the pretrained EfficientNet-B3 model, originally trained on the ImageNet dataset. ImageNet comprises over 14 million labelled images across 1,000 classes, allowing the model to learn a comprehensive set of features, from low-level edges to high-level object representations [29]. This extensive pretraining equips the model with robust feature extraction capabilities, which can be transferred to the task of neuronal spike classification. The training process during the pretraining phase can be mathematically represented as:

$$\theta_{frozen} = \theta_{ImageNet} \quad 13$$

$$\theta_{train} = \underset{\theta}{\operatorname{argmin}} L_{\theta}(f(x; \theta_{frozen}, \theta)) \quad 14$$

where, θ_{frozen} represents the frozen weights from the ImageNet pretraining, θ_{train} represents the trainable weights, $\underset{\theta}{\operatorname{argmin}} L_{\theta}$ is the loss function and $f(x; \theta_{frozen}, \theta)$ is the output of the model.

In the ITSC framework, the fully connected layers of the EfficientNet-B3 model, which are responsible for ImageNet classification, are removed. The intermediate layers, rich in learned features, are retained and serve as feature extractors for the spike images. This approach ensures that the model can effectively utilize the pretrained features to capture the intricate patterns present in the spike waveforms (Refer: Figure 8 for detailed workflow of a spike image).

2.6.3. Fine-Tuning the Model

To adapt EfficientNet-B3, augmented with a Convolutional Block Attention Module (CBAM), for the specific task of classifying neuronal spikes, the original classification head is replaced with a new fully connected layer. This new layer is tailored to the requirements of the ITSC, with the number of output neurons set to 1000, corresponding to the number of distinct

neuron classes in the dataset (Refer Annex: Class Selection criteria). This modification enables the model to output predictions that are directly relevant to the classification of neuronal spikes, leveraging the specialized feature extraction capabilities of EfficientNet-B3 and the attention mechanism implemented using CBAM (Refer Annex: CBAM) [30].

During the initial training phase, the layers of EfficientNet-B3 are frozen, meaning their weights are not updated. This step retains the rich feature representations learned during the pretraining on ImageNet, ensuring that the model begins with a strong foundation for feature extraction. By freezing these layers initially, the attention mechanisms remain intact, enhancing the network's ability to identify important features in the spike images. Freezing the pretrained layers allows the model to leverage the powerful features already learned, reducing the need for extensive retraining and preventing the model from losing its generalization capabilities (Refer Annex: Model Fine-tuning process).

2.6.4. Training and Optimization

The model deals with ~445K spike images of size 224×224. This calls for extensive optimizations and precise training techniques to ensure efficient and effective learning. To optimize the training process, several advanced techniques were employed such as enhancing the memory, optimization of the code, enabling GPU resources among others (Refer Annex: Model Optimization).

Table 5: Overall Model Optimizations

DataLoader Improvements	~10-20% reduction in time.
Mixed Precision + CuDNN Benchmarking	~15-25% reduction in time.
Gradient Accumulation	May slightly increase or decrease time per epoch, depending on the specific hardware and data.
Overall Estimate	Training time per epoch could be reduced by 30-45%.

As a result of these optimizations (Refer Annex: Model Optimization), the training time per epoch was reduced from 41 minutes to approximately 20 minutes, effectively reducing the training duration by half. This significant improvement not only made the training process more efficient but also allowed for more rapid experimentation and fine-tuning of the model. The combination of these advanced techniques ensured that the model could handle the large-scale spike data effectively, leading to faster convergence and better overall performance.

3. Results

3.1. SC Model Results

The SC model directly deals with the raw neuronal signals recorded at 60,000 Hz and is the input to the 1D Resnet-Inspired Model. The dataset is highly imbalanced and hence, underrepresented classes are assigned higher weights during training to avoid bias.

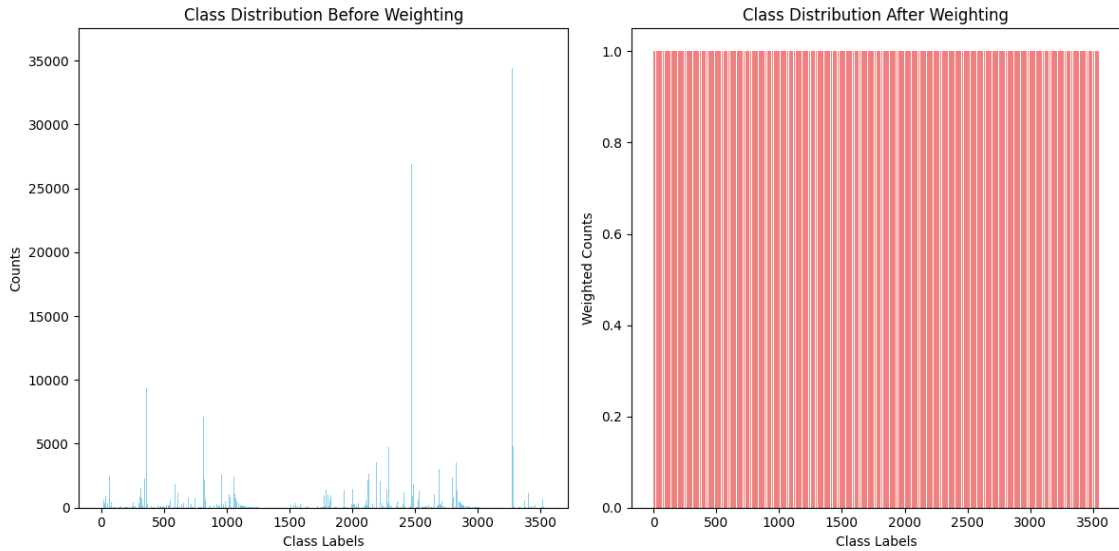


Figure 9: Dealing of class imbalance by assigning higher weights to underrepresented samples in certain classes to avoid bias during training. This process is done by using class weights. There are other methods to deal with class imbalance such as resampling, data augmentation. For this scenario, class weights is the best approach without changing the dataset.

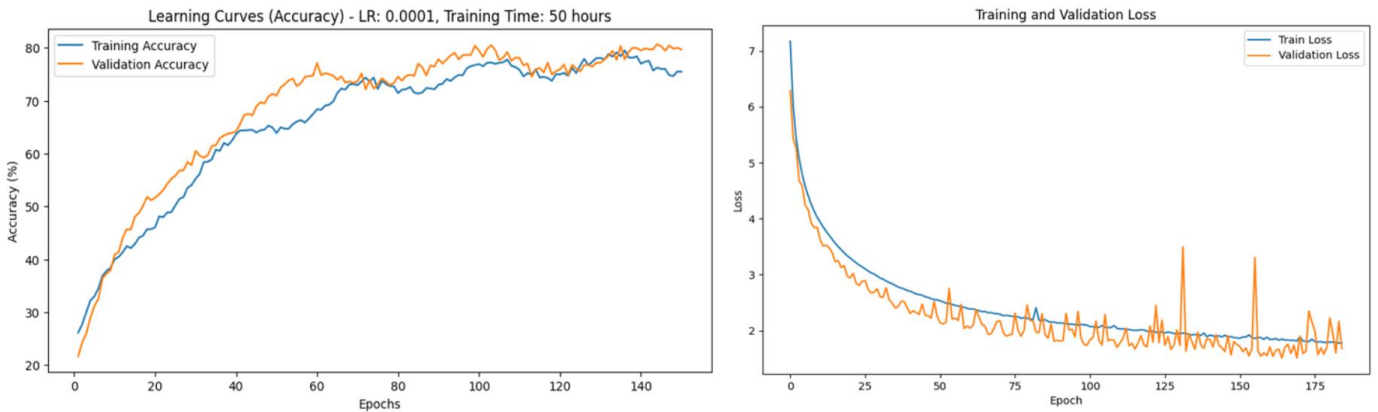


Figure 10: The learning curves of the SC model. The model achieves a validation accuracy of 78.35% on classifying spikes into 1000 distinct neuron classes/labels. The accuracy curves are plotted for 140 epochs requiring 50 hours of training time. The validation loss of 1.1023 is achieved. The loss curves are plotted for 175 epochs to visualize the minimum loss of the model. Although, after epoch 180, there was no change in loss and after epoch 150, the learning curves plateaued. Hence showing that the maximum accuracy the SC model reaches is at 78.35%.

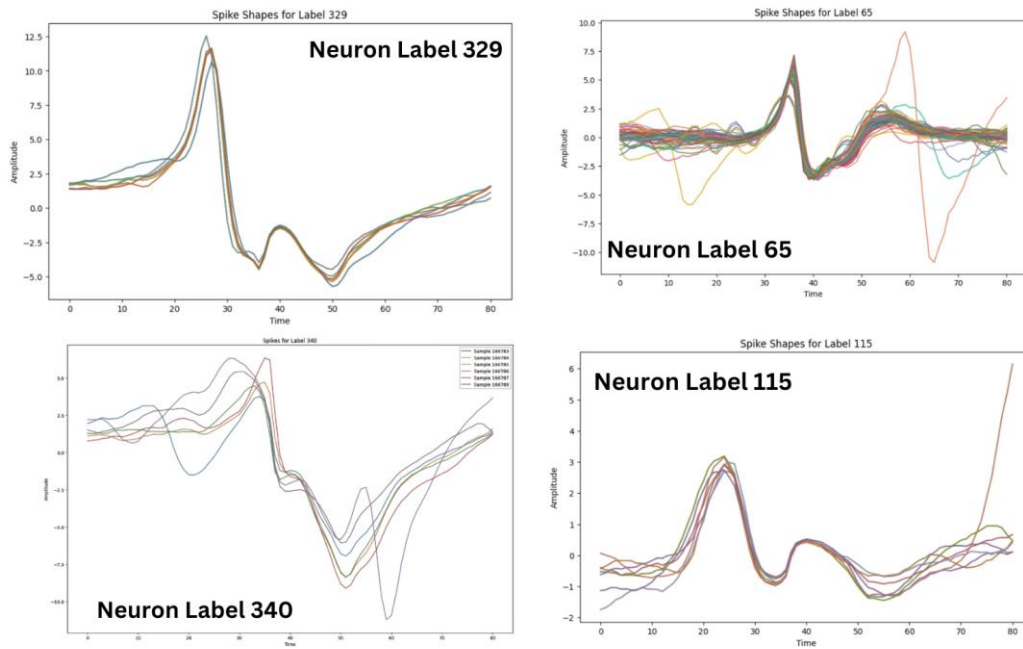


Figure 11: Results of the SC model. The correctly classified spikes with their neuron label. Randomly plotted, these graphs showcase the variability in neural spike shapes, which is relevant for understanding how different neurons respond to stimuli. Each graph corresponds to a specific neuron output (Neuron Label 329, Neuron Label 65, Neuron Label 340, and Neuron Label 115). The x-axes represent time in milliseconds, and the y-axes indicate voltage in microvolts.

3.2. ITSC Model Results

The ITSC model converts the raw neuronal signals to greyscale images of 224x224 standard size and then leverages transfer learning of pretrained model EfficientNet-B3 along with an attention mechanism of the CBAM to accurately classify the spike shapes into 1000 distinct classes.

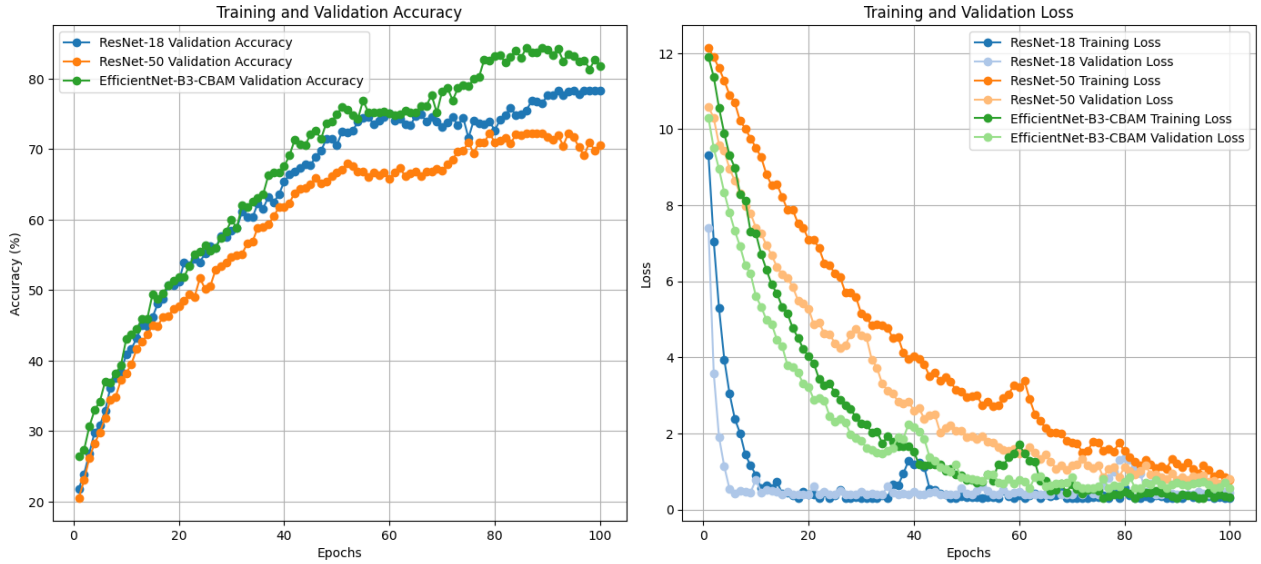


Figure 12: A comparative of the Learning Curves of the Pretrained Models for the ITSC model. The ResNet-18 achieves an accuracy of 67.12% for 15 hours of training. The ResNet-50 has an accuracy of 70.34% and is computationally expensive with 25 hours of training. The Efficientnet-B3, with 10 hours of training achieves the highest accuracy of 86.35%. Hence making EfficientNet-B3 the most efficient implementation of the ITSC model, with reduced training time and higher accuracy. The left of the figure are the loss curves of the respective models. The aim is to achieve the highest accuracy with the least loss values of 0.2367. The highest accuracy of the model is 86.35% with least possible loss of 0.2367.

Table 6: The overall comparison of the Pretrained Model Performance

Model	Accuracy (1000 classes)	Number of Epochs	Training Time (Hours)	Description
ResNet-18	67.12%	100	15	A standard model for comparison, demonstrating good performance with lower computational requirements.
ResNet-50	70.34%	100	25	Deeper architecture provides better accuracy but at the cost of increased training time and resources.
EfficientNet-B3 with CBAM	86.35%	50	10	Combines efficiency and performance, making it ideal for image-based tasks with high accuracy.

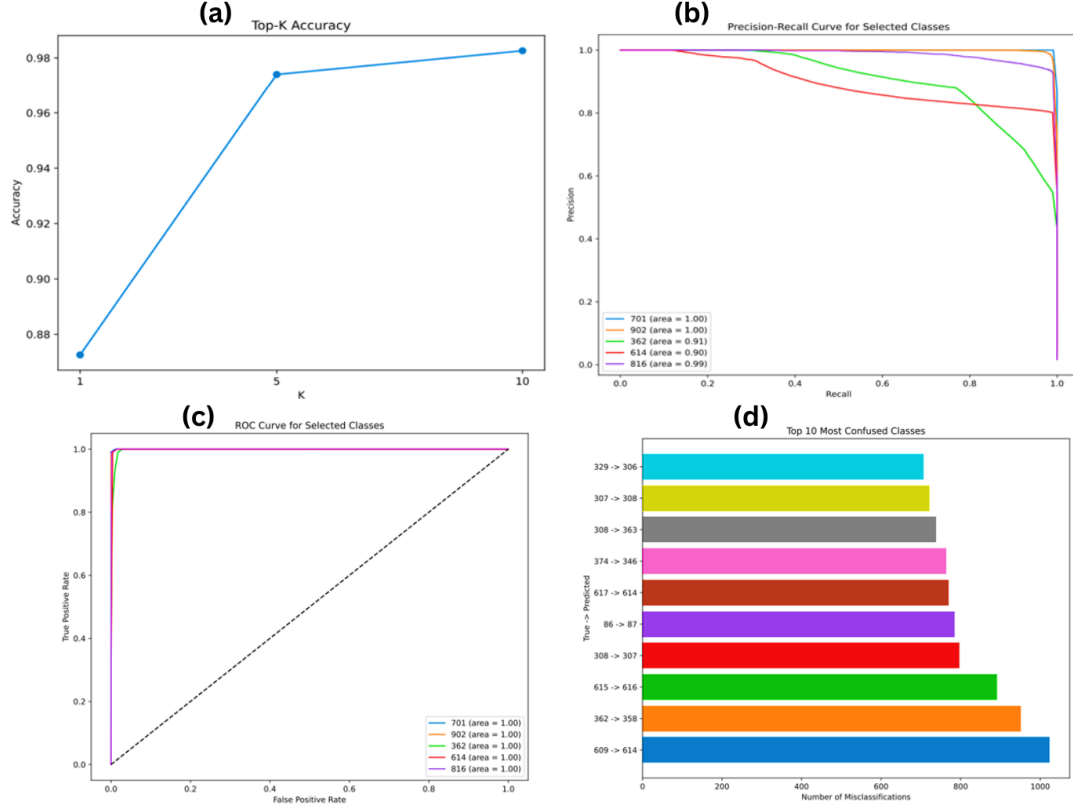


Figure 13: The (a) The Top-K Accuracy graph illustrates the performance of the ITSC model across different values of K. The graph demonstrates a sharp increase in accuracy from just above 0.90 at K=1 to nearly 0.96 at K=2. As K increases from 2 to 10, the accuracy continues to rise gradually, reaching just below 0.98 at K=10. This indicates that the ITSC model’s performance improves significantly when allowed more top predictions.

The (b) Precision-Recall Curves for Selected Classes in ITSC Model: The graph illustrates the precision-recall performance for four selected classes. Classes 701 and 902 achieve perfect precision and recall (AUC = 1.00), while classes 614 and 816 also demonstrate high performance with AUC values of 0.90 and 0.99, respectively. This highlights the ITSC model’s robust classification capabilities across different classes

The (c) ROC Curve for Selected Classes in ITSC Model: The graph illustrates the ROC curves for five selected classes, each achieving an AUC of 1.00. This indicates perfect classification performance, with the model effectively distinguishing between true positives and false positives for each class.

The (d) Top 10 Most Confused Classes in ITSC Model: The graph highlights the pairs of classes most frequently misclassified by the ITSC model. Identifying these patterns provides valuable insights for Refining the model and enhancing classification accuracy. The pairs 609 -> 614 have the highest number of misclassifications, indicating that these classes are often confused with each other.

The above table shows that EfficientNet-B3 has the highest accuracy and least training time and is hence more suitable for the ITSC model implementation (Table 6). The loss curves of all the models do optimize but the accuracy differs. This shows that all the models converge well but the size of each model determines the how well the model is responding to the complexity of the spike data. For model size information refer Table 3: Pretrained Model Selection.

Table 7: The Pretrained Model Top-k accuracy Comparison

Accuracy	ResNet-18	ResNet-50	EfficientNet-B3
Top-1	0.62	0.65	0.77
Top-5	0.80	0.83	0.83

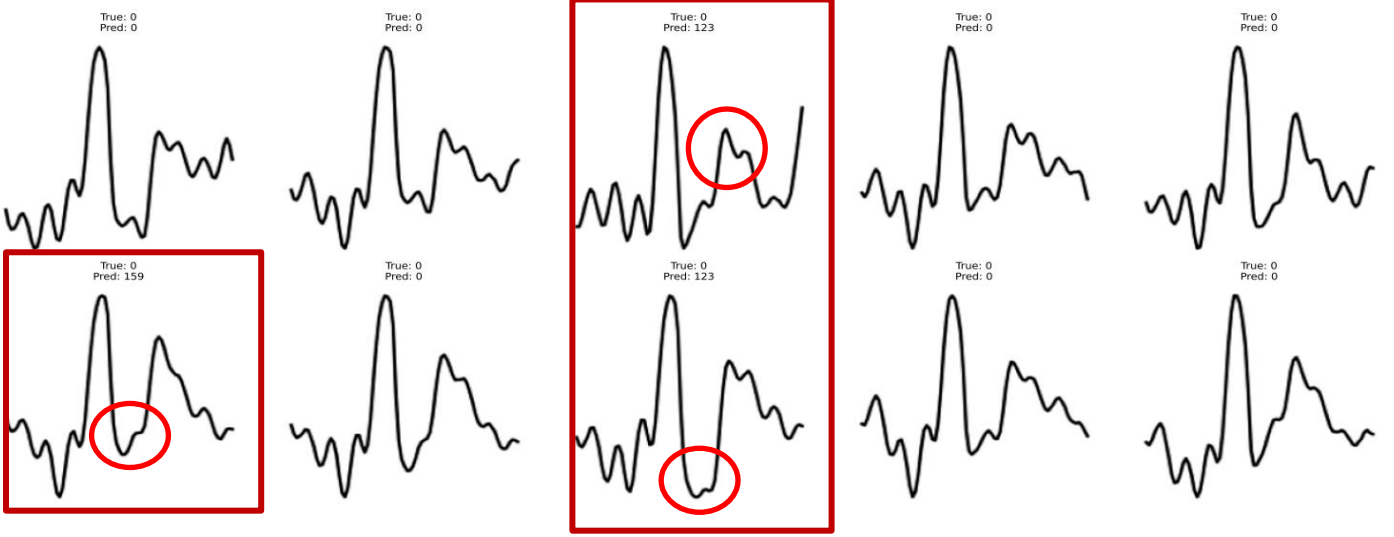


Figure 14: The output of the ITSC model. The highlighted frames are the misclassified spike that belong to other neuron classes. All the spikes look similar but have distinct curves that separate them from other neuron classes/labels.

To evaluate all the models, the below formulas were used:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad 15$$

where TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives, respectively.

$$Precision = \frac{TP}{TP+FP} \quad 16$$

precision, indicating how many of the predicted spikes are correctly classified.

$$Recall = \frac{TP}{TP+FN} \quad 17$$

recall/reflecting the model's ability to identify all relevant spikes. The F1-score is the harmonic mean of precision and recall, computed as :

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad 18$$

providing a balanced measure of accuracy, especially when dealing with imbalanced datasets. These metrics collectively offer a comprehensive view of the model's performance in spike classification tasks.

3.3. ISC Model Results

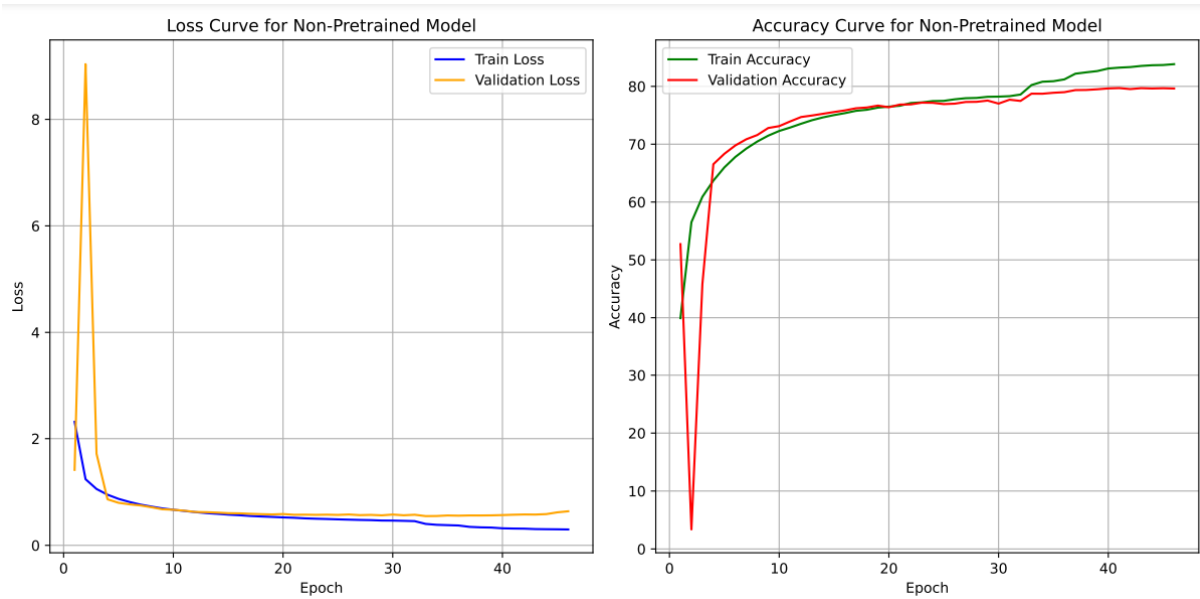


Figure 15: The ISC model learning curves show the accuracy of the ISC model is 79.7% and after that the model begins to overfit. Minimum loss of 0.2367 is achieved, similar to the ITSC but the accuracy is higher for the ITSC model. The overall training time for the ISC model is 50 hours, similar to the SC model with a slightly higher performance accuracy. The ISC model graphs provide a baseline to prove the importance of transfer learning and how it improves performance for significantly lesser training time.

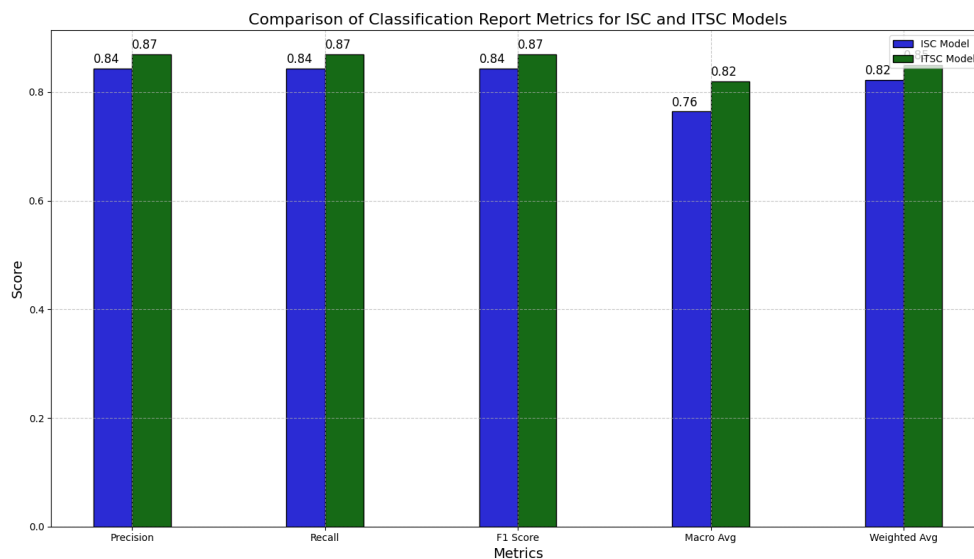


Figure 16: Bar chart comparison of classification report metrics for ISC and ITSC models, showcasing precision, recall, F1 score, macro avg, and weighted avg. The chart visually represents the performance measures of two different models (ISC and ITSC) across various metrics. ITSC generally scores slightly higher than ISC. The height of each bar represents the score for that metric. Scores range from 0.0 to 1.0, with higher values indicating better performance. The ITSC model has higher precision, recall, and F1 score compared to the ISC model. Based on these metrics, the ITSC model appears to perform slightly better overall, especially in terms of precision and recall.

3.4. Final Comparison

The final objective of the project was to provide a comparative study of all the proposed methods for multi-class spike classification. The performance comparison of the three models—Spike-based ResNet Classifier (SC), Image-based Transfer Learning Spike Classifier (ITSC), and Image-based Spike Classifier Non-pretrained (ISC)—reveals distinct differences in their effectiveness for multi-class neuronal spike classification.

Table 8: Summary of Computations across the 3 models

Model	Classification	Accuracy (1000 classes)	Number of Epochs	Training Time (hours)	Justification
Spike-based Classifier (SC)	1D ResNet-18	78.3%	150	50	ResNet-18 is effective for time series data, providing a solid baseline with manageable complexity.
Image-based Transfer Learning Spike Classifier (ITSC)	Pretrained EfficientNet-B3 with CBAM	86.35%	50	10	EfficientNet-B3 with CBAM leverages transfer learning, offering superior accuracy and efficient training.
Image-based Spike Classifier Non-pretrained (ISC)	Non-pretrained EfficientNet-B3 with CBAM	79.75%	100	50	Training EfficientNet-B3 from scratch shows potential but requires more training time to match pre-trained models.

The SC model, which processes raw spike waveforms using a ResNet-inspired architecture, achieved a moderate accuracy of 78%, indicating its capability to handle time-series data but with limitations in capturing complex spike patterns. The ITSC model, which leverages transfer learning with the EfficientNet-B3 architecture and CBAM attention mechanism, significantly outperformed the SC model, achieving an accuracy of 86.35%. This improvement demonstrates the advantage of using pretrained models and attention mechanisms to enhance feature extraction and focus on critical aspects of spike waveforms. In contrast, the ISC model, trained from scratch using the EfficientNet-B3 architecture without pretrained weights, required extensive training and yet achieved an accuracy of 79.7%. Overall, the ITSC model proved to be the most effective, combining high accuracy with computational efficiency.

For the classification task involving 3,544 classes (Refer Table 9: The comparison of all the proposed methods for 3544 classes), the results highlight the varying effectiveness of the three models under study, each trained for 80 hours over 200 epochs. Despite the increased complexity and longer training times required for such many classes, the Image-based Transfer Learning Spike Classifier (ITSC) model, which utilizes a pretrained EfficientNet-B3 with a CBAM attention mechanism, achieved the highest accuracy of 58.39%. This performance surpasses both the Spike-based ResNet Classifier (SC), which managed an accuracy of 34.83%, and the Image-based Spike

Classifier Non-pretrained (ISC), which reached 42.70%. The superior performance of the ITSC model underscores the advantages of leveraging transfer learning and attention mechanisms to refine feature extraction and enhance focus on the most informative aspects of spike waveforms. While the ITSC model requires additional fine-tuning, its ability to better handle the complexity of multi-class classification demonstrates significant promise for further improving accuracy in spike sorting tasks.

Table 9: The comparison of all the proposed methods for 3544 classes

Method	Classification	Accuracy (3544 classes)	Training Time (hours)	Epochs	Limitations
Spike-based Classifier (SC)	1D ResNet-18	34.83%	80	200	Adaptation for time series data
Image-based Spike Classifier Non-pretrained (ISC)	Non-pretrained EffcientNet-B3 with CBAM	42.70%	80	200	Requires large training time
Image-based Transfer Learning Spike Classifier (ITSC)	Pretrained EfficientNet-B3 with CBAM	58.39%	80	200	Fine-tuning required

4. Discussion

This project explored the application of deep learning techniques for multi-class spike classification to address the limitations of traditional spike sorting methods. Specifically, the study aimed to develop and compare three different models: (a) a customized ResNet-18 model for time-series spike waveform classification (SC), (b) an EfficientNet-B3 model trained from scratch for image-based spike classification (ISC), and (c) a pre-trained EfficientNet-B3 model with attention mechanisms for image-based spike classification (ITSC). The results demonstrate that leveraging image processing (H1) and transfer learning (H2) significantly enhances spike classification accuracy, supporting both hypotheses (Refer 1. Introduction for hypothesis) and achieving the project's objectives.

The ITSC model, which employs a pre-trained EfficientNet-B3 architecture combined with a CBAM attention mechanism, achieved the highest accuracy of 86.35% after 50 epochs and 10 hours of training. **This superior performance demonstrates the efficacy of transfer learning in spike classification tasks, supporting the hypothesis (H2) that deep learning and transfer learning can improve classification accuracy and efficiency.** The ITSC model's success highlights the benefits of using a pre-trained model that leverages knowledge from large-scale image datasets, allowing for rapid convergence and high accuracy with minimal additional training. The attention mechanism further enhances this model's capability by focusing on the most relevant features of the spike images, thereby improving its ability to distinguish between different neuronal classes. **This outcome aligns with the hypothesis (H1) that image processing techniques, particularly when augmented with deep learning and attention mechanisms, improve spike classification performance.**

The SC model, which utilizes a ResNet-18 architecture for direct time-series classification, achieved an accuracy of 78.3% over 150 epochs and 50 hours of training. While this model performed well in handling raw waveform data, its lower accuracy relative to the ITSC model suggests limitations in feature extraction capabilities inherent to one-dimensional convolutional networks. The ResNet-18 model provides a solid baseline by effectively managing the complexity of time-series data but lacks the depth and nuance in feature representation that image-based models achieve. The performance gap between the SC and ITSC models (Refer 2.6.1. Conversion of Spike Waveforms to Images) underscores the advantages of converting spike waveforms into images, allowing the use of more sophisticated image-based architectures for better classification accuracy.

The ISC model, trained from scratch using an EfficientNet-B3 architecture with CBAM, achieved an accuracy of 79.75% after 100 epochs and 50 hours of training. While this model outperformed the SC model, it did not reach the level of accuracy achieved by the ITSC model, highlighting the challenges of training deep networks without the benefit of transfer learning. The ISC model's performance indicates that, although it has the potential to learn from high-dimensional spike image data, the lack of pre-trained weights requires more extensive training time and computational resources to achieve comparable results. This finding suggests that while training models from scratch can provide valuable insights, leveraging transfer learning remains a more efficient and effective strategy for complex classification tasks like spike sorting.

While the ITSC model shows the most promise, it is important to note that it also has limitations. The need for fine-tuning pre-trained models can introduce complexity, requiring careful selection of hyperparameters and extensive computational resources. Additionally, the reliance on image conversion might obscure some temporal features inherent in the original spike waveforms, potentially leading to a loss of critical information. Future work could explore hybrid approaches that integrate both time-series and image-based data to capture a more complete representation of neuronal activity.

In comparison to existing literature, the findings from this study demonstrate a marked improvement in spike classification accuracy using deep learning techniques. Traditional methods, such as PCA and wavelet transforms combined with clustering algorithms, typically achieve accuracies ranging from 70-80% but often rely on manual intervention and are sensitive to noise and variability in spike shapes [1] [3]. In contrast, the ITSC model's approach using transfer learning and attention mechanisms offers a more automated and robust solution, capable of handling a larger number of neuron classes with improved accuracy. These findings suggest that the application of deep learning and transfer learning in neuroscience can significantly enhance the accuracy and efficiency of spike classification, potentially setting a new benchmark for future research.

Despite the promising results, several limitations must be acknowledged. The dataset used in this study, while comprehensive, is derived from a single source (dragonflies) and may not fully represent the diversity of neuronal activity across different species or recording conditions. Additionally, the models were trained on a subset of 1,000 classes due to computational constraints, which may have affected the generalizability of the findings to datasets with a larger number of neuron classes. Future research could address these limitations by incorporating more diverse datasets and optimizing the models for larger-scale classification tasks.

Moreover, while the ITSC model achieved the highest accuracy, the overall performance suggests that there is still room for improvement. For instance, incorporating temporal dynamics more effectively into the image-based models or exploring ensemble methods that combine different deep learning architectures could further enhance classification accuracy. Additionally, advancements in hardware acceleration and distributed training techniques could reduce the computational burden associated with fine-tuning large models, enabling the exploration of more complex architectures.

In conclusion, this study successfully demonstrated that deep learning techniques, particularly transfer learning with EfficientNet-B3 and attention mechanisms, significantly improve spike classification accuracy compared to traditional methods. The findings support the proposed hypotheses (H1 and H2) and suggest that further research could refine these models to achieve even higher accuracy and generalizability, contributing to a more comprehensive understanding of neuronal behaviour and activity.

5. References

- [1] M. S. Lewicki, "A review of methods for spike sorting: the detection and classification of neural action potentials," *Network: Computation in Neural Systems*, pp. 9(4), R53-R78, 1998.
- [2] J. H. e. a. Lee, "Deep learning-based spike sorting algorithm for multi-unit recordings," *Journal of Neuroscience Methods*, no. 291, pp. 1-9, 2017.
- [3] R. Q. N. Z. & B.-S. Y. Quiroga, "Unsupervised spike detection and sorting with wavelets and superparamagnetic clustering," *Neural Computation*, Vols. 16(8), 1661-1687, 2004.
- [4] H. T. W. C. H. & C. Y. Shen, "Deep convolutional network based spike sorting.," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 27(6), pp. 1218-1228, 2019.
- [5] Y. W. Z. N. Y. Z. T. Z. S. L. Y. S. P. & W. Q. Kong, "Unlocking the Power of LSTM for Long Term Time Series Forecasting," arXiv preprint, 2023. [Online].
- [6] E.-R. C. A. I. A.-M. D. M. & M. R. C. Ardelean, "A study of autoencoders as a feature extraction technique for spike sorting," *PLoS ONE*, 18(3), vol. e0282810, 2023.
- [7] T. C. M. P. & K. N. Kietzmann, "Deep Neural Networks in Computational Neuroscience," *Oxford Research Encyclopedia of Neuroscience*, 2019.
- [8] D. L. H. H. C. C. F. S. E. A. S. D. & D. J. J. Yamins, "Performance-optimized hierarchical models predict neural responses in higher visual cortex.," *Proceedings of the National Academy of Sciences*, vol. 111(23), pp. 8619-8624, 2014.
- [9] A. & M. A. Tavanaei, "Deep learning in spike sorting: Opportunities and challenges.," *Neural Networks*, 118, pp. 278-284, 2019.
- [10] D. L. Donoho, "De-noising by soft-thresholding," *IEEE Transactions on Information Theory*, pp. 41(3), 613-627, 1995.
- [11] M. S. M. P. P. & K. D. Fee, "Automatic sorting of multiple unit neuronal signals in the presence of anisotropic and non-Gaussian variability," *Journal of Neuroscience Methods*, pp. 69(2), 175-188, 1996.
- [12] Z. L. a. V. Y. F. Tan, "The Informativeness of k-Means for Learning," *arXiv:1703.10534v1 [stat.ML]*, 30 March, 2017.
- [13] F. B. M. J. & V.-I. C. Wood, "On the variability of manual spike sorting," *IEEE Transactions on Biomedical Engineering*, 51(6), 912-920, 2004.
- [14] J. Y. T. P. B. A. A. (. T. i. D. E. M. f. I. S. Maruf Gögebakan Hemanth, "Gaussian Mixture Model-Based Clustering of Multivariate Data Using Soft Computing Hybrid Algorithm," in *Lecture Notes on Data Engineering and Communications Technologies*, vol 76. Springer, Cham. https://doi.org/10.1007/978-3-030-79357-9_49, ICAIAME 2020.
- [15] M. V. R. & B. A. Ul Hassan, "Efficient neural spike sorting using data subdivision and unification.," *PLOS ONE*, vol. 16(2), no. e0245589., 2021.
- [16] M. R. et.al, "Online spike sorting via deep contractive autoencoder," *Neural Networks*, vol. Vol 155, no. <https://doi.org/10.1016/j.neunet.2022.08.001>, pp. Pages 39-49, 2022.
- [17] K. Z. X. R. S. & S. J. He, "Deep residual learning for image recognition.," *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. (pp. 770-778), (2016). .
- [18] Y. B. a. G. H. Y. LeCun, "Deep learning," *Nature*, Vols. vol. 521, no. 7553, pp. pp. 436-444, 2015.

- [19] X. Z. S. R. a. J. S. K. He, "Deep residual learning for image recognition," *Proc. IEEE Conf. Comput. Vis. Pattern Recognit*, pp. pp. 770-778, 2016.
- [20] C. M. Bishop, "Pattern Recognition and Machine Learning.," *USA: Springer, New York, NY*, 2006.
- [21] D. P. K. a. J. Ba, "Adam: A method for stochastic optimization," 2015.
- [22] D. P. & B. J. Kingma, "Adam: A method for stochastic optimization.," 2015. [Online]. Available: arXiv preprint arXiv:1412.6980..
- [23] K. Z. X. R. S. & S. J. He, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification.," *Proceedings of the IEEE international conference on computer vision*, pp. pp. 1026-1034, 2015.
- [24] X. & B. Y. Glorot, "Understanding the difficulty of training deep feedforward neural networks.," *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. pp. 249-256, 2010.
- [25] Y.-C. B. Amreen Batool, "Lightweight EfficientNetB3 Model Based on Depthwise Separable Convolutions for Enhancing Classification of Leukemia White Blood Cell Images," *IEEE*, 2023.
- [26] Q. V. L. Mingxing Tan, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *International Conference on Machine Learning*, 2019, arXiv:1905.11946 [cs.LG].
- [27] M. & L. Q. V. Tan, "EfficientNet: Rethinking model scaling for convolutional neural networks.," *In International Conference on Machine Learning.*, pp. (pp. 6105-6114). PMLR, 2019.
- [28] J. Brownlee, "8 Tactics to Combat Imbalanced Classes in Your Machine Learning Dataset," *MachineLearningMastery.com.*, 2020. [Online].
- [29] W. D. R. S. L.-J. L. K. L. a. L. F.-F. J. Deng, "ImageNet: A large-scale hierarchical image database," *Proc. IEEE Conf. Comput. Vis. Pattern Recognit*, pp. pp. 248-255, 2009.
- [30] S. P. J. L. J. Y. & K. I. S. Woo, "CBAM: Convolutional block attention module.," *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. pp. 3-19, 2018.
- [31] P. M. Antoine Nonclercq, "Quantification of Motion Artifact Rejection Due to Active Electrodes and Driven-Right-Leg Circuit in Spike Detection Algorithms," in *IEEE Transactions on Biomedical Engineering*, 2010.
- [32] J. D. W. S. R. L. L.-J. L. K. & F.-F. L. Deng, "ImageNet: A Large-Scale Hierarchical Image Database.," in *In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.
- [33] S. J. & Y. Q. Pan, "A survey on transfer learning.," *IEEE Transactions on Knowledge and Data Engineering.*, vol. 22(10), pp. 1345-1359, 2010.
- [34] J. C. J. B. Y. & L. H. Yosinski, "How transferable are features in deep neural networks?," *In Advances in Neural Information Processing Systems.*, pp. (pp. 3320-3328), 2014.
- [35] A. S. I. & H. G. E. Krizhevsky, "ImageNet classification with deep convolutional neural networks.," *NeurlPS Proceedings, part of: Advances in neural information processing systems*, vol. 25, pp. 1097-1105, 2012.
- [36] Y. C. A. & V. P. Bengio, "Representation learning: A review and new perspectives.," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35(8), pp. 1798-1828, 2013.

- [37] A. G. Z. M. C. B. K. D. W. W. W. T. .. & A. H. Howard, “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.,” *arXiv preprint*, no. arXiv:1704.04861, 2017.
- [38] L. S. a. G. S. J. Hu, “Squeeze-and-Excitation Networks,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, no. doi: 10.1109/CVPR.2018.00745, pp. pp. 7132-7141, 2018.
- [39] P. Z. B. & L. Q. V. Ramachandran, “Searching for activation functions,” *arXiv preprint*, no. arXiv:1710.05941, 2017.
- [40] F. Q. e. al, “Micro-expression Recognition Based on DCBAMEfficientNet Model,” *J. Phys.*, vol. Conf. Ser. 2504 012062, 2023.
- [41] P. N. S. A. J. D. G. E. E. G. D. & H. M. Micikevicius, “Mixed Precision Training.,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.
- [42] S. W. C. V. P. C. J. T. J. C. B. & S. E. Chetlur, “cuDNN: Efficient Primitives for Deep Learning.,” *arXiv preprint arXiv:1410.0759*, 2014.
- [43] N. S. M. D. N. J. S. M. & T. P. T. P. Keskar, “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima.,” *arXiv preprint arXiv:1609.04836*., 2016. [Online].
- [44] D. P. & B. J. Kingma, “Adam: A Method for Stochastic Optimization.,” in *In Proceedings of the 3rd International Conference on Learning Representations (ICLR)*., 2015.
- [45] F. J. Z. & O. K. Franke, “An automated spike sorting algorithm on the basis of the optimal success rate.,” *PLOS ONE*, 10(6), e0130247, 2020.
- [46] J. D. W. S. R. L. L. J. L. K. & F.-F. L. Deng, “ImageNet: A large-scale hierarchical image database.,” *CVPR (pp. 248-255)*, 2015.
- [47] Y. E. J. K. & E. J. Zhang, “The impact of transfer learning on deep neural networks for spike sorting.,” *Frontiers in Neuroscience*, 17., 2023.
- [48] F. C. a. S. M. B. B. Yin, “Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks,” *arXiv*, 2021.
- [49] M. G. S. R. K. T. M. a. A. S. M. A. Tavanaei, “Deep learning in spiking neural networks,” *Neural Networks*, 2019.
- [50] R. S. Misu, “Brain Tumor Detection Using Deep Learning Approaches,” *arXiv*, 2023.
- [51] M. W. E. N. X. W. G. L. G. D. M. B. D. S. J. a. W. L. J. K. Eshraghian, “Training Spiking Neural Networks Using Lessons From Deep Learning,” *Proceedings of the IEEE*, 2023.
- [52] G. S. P. P. a. K. R. N. Rathi, “Enabling Deep Spiking Neural Networks with Hybrid Conversion and Spike Timing Dependent Backpropagation,” *arXiv*, 2020.
- [53] F. C. a. S. M. B. B. Yin, “Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks,” *arXiv*, 2021.
- [54] C. v. G. R. D. R. C. a. J. d. R. M. A. Uran, “Applying Transfer Learning To Deep Learned Models For EEG Analysis,” *arXiv*, 2019.
- [55] L. Q. G. R. W. Z. Q. F. Zhenhua Deng, “From Early Domesticated Rice of the Middle Yangtze Basin to Millet, Rice and Wheat Agriculture: Archaeobotanical Macro-Remains from Baligang, Nanyang Basin, Central China (6700–500 BC),” 2015. [Online]. Available: <https://doi.org/10.1371/journal.pone.0139885>.
- [56] M. Schmitt, “Deep learning in business analytics: A clash of expectations and reality,” *Science Direct*, 2022.

- [57] X. Z. S. R. J. S. Kaiming He, “Deep Residual Learning for Image Recognition,” in *arXiv:1512.03385*, University of Cornell, 2015.
- [58] R. V. B. Masood Ul Hassan, “Efficient neural spike sorting using data subdivision and unification,” *Plos One*, no. <https://doi.org/10.1371/journal.pone.0245589>, February 10, 2021.
- [59] Y. H. X. G. P. W. Y. Z. Wentao Quan1, “A comprehensive review of spike sorting algorithms,” *Progress in Medical Devices- Zentime*, June 30, 2024.
- [60] Y. Q. a. G. P. Hang Yu, “ NeuSort: An Automatic Adaptive Spike Sorting,” *arXiv:2304.10273v1 [cs.NE] 20 Apr 2023*.
- [61] X. B. A. & B. Y. Glorot, “ Deep sparse rectifier neural networks,” In *Proceedings of the fourteenth international conference on artificial intelligence and statistics .*, pp. (pp. 315-323).
- [62] L. B. Y. B. a. P. H. Y. LeCun, “Gradient-based learning applied to document recognition,” *Proc. IEEE no. 11*, vol. vol. 86, pp. pp. 2278-2324, Nov. 1998.
- [63] Y. B. a. G. H. Y. LeCun, “Deep learning,” *Nature*, vol. vol. 521, no. no. 7553, pp. pp. 436-444, 2015.
- [64] Y. B. a. A. C. I. Goodfellow, “Deep Learning,” in *MIT Press*, Cambridge, MA, USA, 2016.
- [65] M. Z. C. K. J. & B. S. Raghu, “Transfusion: Understanding transfer learning for medical imaging,” *Advances in Neural Information Processing Systems* , pp. (pp. 3342-3352)., 2019.
- [66] G. E. & S. R. R. Hinton, “ Reducing the dimensionality of data with neural networks.,” *Science*, 313(5786)., pp. 504-507, 2006.
- [67] J. G. R. & F. A. Xie, “ Unsupervised deep embedding for clustering analysis.,” *International Conference on Machine Learning*, pp. (pp. 478-487)., 2016.
- [68] P. U. & C. M. Diehl, “ Unsupervised learning of digit recognition using spike-timing-dependent plasticity.,” *Frontiers in Computational Neuroscience*, Vols. 9, 99, 2015.
- [69] P. e. a. Yger, “A spike sorting toolbox for up to thousands of electrodes validated with ground truth recordings in vitro and in vivo.,” *eLife*, vol. e34518, no. 7, 2018.
- [70] R. X. Y. Z. Y. H. X. L. Y. & Z. S. Wang, “A Fast and Effective Spike Sorting Method Based on Multi-Frequency Composite Waveform Shapes,” *Brain Sciences*, vol. 13(8), no. 1156, 2023.
- [71] Y. W. N. Z. a. X. L. Z. Li, “An Accurate and Robust Method for Spike Sorting Based on Convolutional Neural Networks,” *Brain Sciences*, vol. 10, no. 11, p. p. 835, November 2020.
- [72] Y. J. X. C. Y. & L. J. Zhou, “ A survey of deep learning techniques applied to visual object detection,” *IEEE Access*, no. 8, pp. 130518-130528, 2020.
- [73] P. S. a. Y. A, “ Non-stationary neural signal to image conversion framework for image-based deep learning algorithms,” *Front. Neuroinform.* 17:1081160., no. doi: 10.3389/fninf.2023.1081160, 2023.
- [74] J. C. L. Z. J. C. W. S. Y. W. Yuan Zhang, “Razor SNN: Efficient Spiking Neural Network with Temporal Embeddings,” [Online]. Available: <https://arxiv.org/abs/2306.17597>.
- [75] Y. Z. et.al, “Razor SNN: Efficient Spiking Neural Network with Temporal Embeddings,” *arXiv*, 2023.

Annex

Spike sorting is a fundamental task in neuroscience, involving the identification and classification of neuronal spikes from extracellular recordings. Signal preprocessing often involves techniques such as filtering, artifact removal, and normalization to improve the signal-to-noise ratio and enhance the distinguishability of the spikes. The feature extraction step then identifies salient characteristics of the spike waveforms, such as their amplitude, duration, and shape, which serve as the basis for subsequent clustering algorithms [31]. The clustering step is particularly important in traditional spike sorting, as it is responsible for grouping the extracted spike features into distinct neuronal units. A variety of clustering algorithms have been employed, ranging from simple threshold-based approaches to more methods such as principal component analysis, k-means, and Gaussian mixture models, discussed in detail in Table 1: Existing Spike Sorting Methodologies.

Transfer Learning

In machine learning, developing models that can perform specific tasks often requires large amounts of labelled data and substantial computational resources. However, in many fields, including neuroscience, acquiring such data can be challenging and costly. To address these challenges, transfer learning has emerged as an effective approach.

Transfer learning involves reusing a model that has already been trained on a large, general dataset for a new, related task. The core idea is that the knowledge gained by the model during its initial training can be applied to solve a different, but related problem more efficiently. For instance, a model pre-trained on ImageNet, a dataset containing millions of labelled images across thousands of categories, has learned to recognize a wide variety of features such as edges, textures, and shapes [32]. This pre-trained model can then be adapted to a new task—such as classifying neuronal spike images—with much less data and training time.

By starting with a model that already knows how to identify basic features, transfer learning can significantly reduce the amount of labelled data and computational power needed for training. This approach is particularly advantageous in scenarios where labelled data is scarce or difficult to obtain, as is often the case in neuroscience [33].

In this study, transfer learning is applied to classify spike waveforms by transforming them into images and using a pre-trained model, EfficientNet-B3, fine-tuned specifically for this task. This technique leverages the powerful feature extraction capabilities of the pre-trained model to improve accuracy and efficiency in spike classification. The knowledge acquired during this initial training phase, encapsulated in the model's weights and learned features, is then transferred to a new model tasked with solving a different, often more specific problem [27]. This process can significantly reduce the amount of data and computational resources required for training, as the model starts with a strong foundation of pre-learned features.

Spike classification, the process of identifying and categorizing neuronal spikes from multi-unit recordings, presents unique challenges. The advent of deep learning has introduced more automated and accurate approaches, but these methods still require substantial amounts of labelled data for effective training. Transfer learning offers a promising solution to these challenges. This approach can enhance the model's ability to recognize and classify complex patterns in neuronal spike data, even with limited labelled examples [34]. Transfer learning typically involves two main stages:

- a) *Pre-training Phase*: The model is initially trained on a large, general dataset, such as ImageNet, which contains millions of labelled images across thousands of categories. During this phase, the model learns to extract general features like edges, textures, and shapes that are common across different tasks and data types [35].
- b) *Fine-Tuning Phase*: The pre-trained model is then adapted to the specific task of interest, such as spike classification. In this phase, the pre-trained weights serve as an initialization point, and the model is fine-tuned using a smaller, task-specific dataset. Only a subset of the model's parameters is typically updated during fine-tuning, while the rest of the parameters remain fixed. This allows the model to quickly adapt to the new task while retaining the general knowledge learned during pre-training.

The fine-tuning process can be mathematically represented as:

$$\theta_{new} = \theta_{pre} - \eta \nabla_{\theta_{pre}} \mathcal{L}(\theta_{pre}, x_{new}, y_{new}) \quad 19$$

where θ_{pre} represents the pre-trained weights, η is the learning rate, $\mathcal{L}$ is the loss function specific to the new task, and x_{new}, y_{new} are the data and labels for the new task.

Transfer learning significantly accelerates the training process for several reasons:

- a) *Reduced Data Requirements*: Transfer learning mitigates the need for large amounts of labelled data for the new task. Since the model already possesses a strong foundation of pre-learned features from the pre-training phase, it requires fewer labelled examples to learn task-specific features effectively [36]. This is particularly advantageous in domains like neuroscience, where obtaining labelled spike data can be resource-intensive and time-consuming.
- b) *Lower Computational Cost*: Because the model starts with weights that are already tuned to recognize general patterns, the number of epochs and computational resources required for training is significantly reduced. The initial training phase, which typically demands high computational power, has already been completed using a large, general-purpose dataset. During fine-tuning, the model only needs to adjust a smaller number of parameters, focusing on refining its knowledge for the new task. This reduction in computational burden can be expressed as:

$$T_{fine-train} \approx \frac{N_{fine-train}}{N_{pre-train}} T_{pre-train} \quad 20$$

where, $T_{fine-train}$ is the time for fine-tuning, $T_{pre-train}$ is the time for pre-training and $N_{fine-train}$ and $N_{pre-train}$ are the numbers of trainable parameters during fine-tuning and pre-training, respectively.

- c) *Improved Convergence Rate*: Models using transfer learning tend to converge faster than models trained from scratch because they start with a set of weights that are already closer to the optimal solution. This faster convergence can be attributed to the fact that the model has already learned a set of low-level and mid-level features that are broadly useful across many tasks. This prior knowledge enables the model to quickly adapt these features to new tasks, like spike classification, with minimal adjustments, thereby reducing the training time significantly [27].

- d) *Better Generalization*: Transfer learning helps prevent overfitting, especially when training on small datasets, by using pre-trained weights as a regularization mechanism. The pre-trained model is less likely to memorize the training data and more likely to generalize well to unseen data because it already captures a wide range of features that are relevant across various tasks [34]. This is crucial in spike classification, where the variability in neuronal signals can lead to overfitting if trained on small, specific datasets.

Class Selection criteria

The use of transfer learning allows for lesser labelled data that can still be used effectively for training and spike classification. Hence for this project the focus was on ~442K spikes, instead of the original ~1.45 million spikes. This decision was driven by several key factors:

- a) *Computational Resources*: Training a model with 3544 classes requires substantial computational power and memory. The complexity of the model increases exponentially with the number of classes, leading to longer training times and higher resource consumption. By reducing the number of classes to 1000, I optimized the use of available computational resources, ensuring a more efficient and manageable training process.
- b) *Training Time*: The training time for a model with 3544 classes was significantly longer, making it impractical for iterative experimentation and fine-tuning. For instance, training EfficientNet-B3 with CBAM for 3544 classes resulted in only 25% accuracy at epoch 24/150, indicating that the model required more epochs and more training time to achieve satisfactory performance (approximately 500 hours of training). In contrast, training the model for 1000 classes yielded 72% accuracy at epoch 10/50, demonstrating a more rapid convergence and allowing for quicker iterations and adjustments in the real-time (10 hours of training).
- c) *Class Imbalance and Data Quality*: With a larger number of classes, the risk of class imbalance increases, where some classes may have significantly fewer samples than others. This imbalance can negatively impact the model's performance, as it may struggle to learn from underrepresented classes. By focusing on 1000 classes, it ensures a more balanced and representative dataset, improving the overall quality and reliability of the training data.
- d) *Model Complexity and Overfitting*: A model trained on 3544 classes is more prone to overfitting, especially if the dataset is not sufficiently large or diverse. Overfitting occurs when the model learns to memorize the training data rather than generalize from it, leading to poor performance on unseen data. By reducing the number of classes, the risk of overfitting is mitigated and enhanced the model's ability to generalize, resulting in more robust and accurate predictions.
- e) *Feasibility and Iterative Development*: Focusing on 1000 classes allowed for a more feasible and iterative development process. It enabled experiments with different model architectures, hyperparameters, and preprocessing techniques more efficiently. This

iterative approach was crucial for refining the model and achieving optimal performance within a reasonable timeframe.

- f) *Scalability and Future Work:* Starting with 1000 classes provided a solid foundation for future scalability. Once the model demonstrated satisfactory performance on this subset, the insights and optimizations gained could be applied to gradually increase the number of classes. This step-by-step approach ensures that the model remains scalable and adaptable to larger datasets in future research.
- g) *Resource Constraints:* Given the constraints of available computational resources and time, it was practical to focus on a smaller subset of classes. This decision allowed for more efficient use of resources, ensuring that the project could be completed within the given limitations while still achieving meaningful and impactful results.

In conclusion, the decision to use 1000 classes instead of 3544 was driven by a combination of technical, computational, and practical considerations. This approach ensured a more efficient, manageable, and effective training process, ultimately leading to a more robust and accurate model for classifying neuronal spikes.

Model Selection

a) Spike-based Classifier (SC) with ResNet-18

The choice of ResNet-18 for the Spike-based Classifier (SC) is driven by its architectural design, which is particularly well-suited for handling one-dimensional (1D) time-series data like raw neuronal spike waveforms. ResNet-18 employs residual learning through skip connections that directly address the vanishing gradient problem—a common issue in deep neural networks where gradients diminish as they are backpropagated through layers [17].

For spike waveform classification, ResNet-18's architecture, which includes both 1D convolutional layers and pooling layers, is advantageous because it allows for the extraction of temporal features across multiple time scales. The 1D convolutional layers apply filters along the temporal axis of the waveform data:

$$h_i^{(l+1)} = f \left(\sum_{k=1}^K w_{i,k}^{(l)} * h_k^{(l)} + b_i^{(l)} \right) \quad 21$$

where $w_{i,k}^{(l)}$ represents the convolutional filter weights, $*$ denotes convolution, $h_k^{(l)}$ the feature map at layer l and $f()$ is the activation function (e.g., ReLU). These operations help capture the essential characteristics of spike waveforms, such as peaks, troughs, and slopes, which are critical for distinguishing different neuronal types.

ResNet-18's relatively low parameter count (~ 11.7 million) and its efficient computational footprint make it an optimal choice for rapid experimentation and model iteration. This model's architecture effectively balances depth and computational efficiency, providing a robust baseline for spike waveform classification without excessive computational overhead.

b) Image-based Transfer Learning Spike Classifier (ITSC) with EfficientNet-B3 and CBAM

The architecture of the EfficientNet-B3 employs a compound scaling method that uniformly scales the network's depth, width, and resolution, leading to better performance and efficiency using compound scaling. This architecture is particularly effective for image-based tasks. It leverages pre-trained weights from large-scale image datasets, enabling it to transfer learned features to the task of neuronal spike classification.

EfficientNet-B3 leverages depth-wise separable convolutions within its Mobile Inverted Bottleneck Convolution (MBConv) blocks, significantly reducing the number of parameters and computational complexity compared to traditional convolutions [37]. The depth-wise separable convolution decomposes a standard convolution operation into a depth-wise convolution followed by a pointwise convolution, effectively reducing the model's parameter count by a factor of:

$$\frac{D_k^2 C_{in} C_{out}}{D_k^2 C_{in} + C_{out}} \quad 22$$

where D_k is the kernel size, C_{in}, C_{out} are the input and output channels, respectively. This operation allows EfficientNet-B3 to extract high-dimensional spatial features efficiently, which is crucial for accurately classifying spike waveforms converted to images.

Additionally, EfficientNet-B3's integration with the Convolutional Block Attention Module (CBAM) further enhances its feature extraction capabilities by focusing on the most salient parts of the spike images. CBAM applies both channel and spatial attention to refine feature maps, enhancing regions that are more indicative of specific spike characteristics. The channel attention mechanism is defined as:

$$M_{ch} = \sigma \left(FC \left(ReLU \left(FC(F_{avg}^{ch}) \right) \right) + FC \left(ReLU \left(FC(F_{max}^{ch}) \right) \right) \right) \quad 23$$

where, F_{avg}^{ch} and F_{max}^{ch} are the feature descriptors obtained by global average pooling and max pooling, respectively, and σ is the sigmoid function. This attention mechanism allows the model to dynamically emphasize channels that are more relevant for the classification task, such as those capturing unique spike patterns or specific neuronal firing characteristics.

This significantly enhances the model's ability to generalize from limited data. The combination of EfficientNet-B3 with CBAM (Convolutional Block Attention Module) enhances the model's ability to focus on important features within the images, leading to improved classification accuracy. Also, EfficientNet-B3 has demonstrated superior performance in various image classification tasks, achieving higher accuracy compared to ResNet-18. EfficientNet-B3 achieved 82% accuracy on ImageNet, compared to 75% for ResNet-18 [27].

EfficientNet-B3

The core innovation of EfficientNet-B3 lies in its compound scaling approach. Unlike traditional CNNs, which typically scale only one dimension (either depth, width, or resolution), EfficientNet-B3 scales all three dimensions using a set of predefined coefficients. This method

ensures that the network can handle higher resolution images with increased model capacity while maintaining computational efficiency.

Key Components of EfficientNet-B3:

- a) *Mobile Inverted Bottleneck Convolution (MBConv) Blocks*: EfficientNet-B3 employs MBConv blocks with depth-wise separable convolutions and squeeze-and-excitation (SE) optimization. The MBConv layers are designed to reduce the number of parameters while preserving model accuracy. Depth-wise separable convolutions split a standard convolution into two operations: a depth-wise convolution and a pointwise convolution. This significantly reduces the computational cost [37].
- b) *Squeeze-and-Excitation (SE) Layers*: Integrated within the MBConv blocks, SE layers adaptively recalibrate channel-wise feature responses by modelling interdependencies between channels. This mechanism helps the network emphasize important features and suppress less relevant ones, enhancing feature learning and improving classification accuracy [38].
- c) *Compound Scaling*: EfficientNet-B3 scales network depth, width, and resolution uniformly according to the formula:

$$Depth = \alpha^d, Width = \beta^d, Resolution = \gamma^d \quad 24$$

where α, β, γ are constants determined by grid search to optimize the scaling of each dimension, and d represents the number of scaling steps. This allows EfficientNet-B3 to maintain high performance on a range of image sizes.

- d) *Activation Function*: The architecture uses the Swish activation function,

$$Swish(x) = x \cdot sigmoid(x) \quad 25$$

which has been shown to improve performance by maintaining smooth gradients and preventing the vanishing gradient problem during backpropagation [39].

With respect to ITSC model, the pretrained EfficientNet-B3 model uses its convolutional layers and MBConv blocks to extract hierarchical features from the spike images. Initial layers capture low-level features such as edges and textures, while deeper layers capture more abstract patterns, such as waveform shapes and firing patterns specific to different neurons.

$$F = MBConv(I) \quad 26$$

Here, I represents the input spike image, and F is the extracted feature map. After feature extraction, the output of EfficientNet-B3 is a set of feature maps that capture spatial patterns related to spike characteristics. These feature maps are then passed through fully connected layers to generate a probability distribution over the predefined neuron classes.

$$\hat{y} = softmax(W + F \cdot b) \quad 27$$

where W and b are the weights and biases of the dense layer, $\hat{y}$ and is the predicted probability distribution.

CBAM

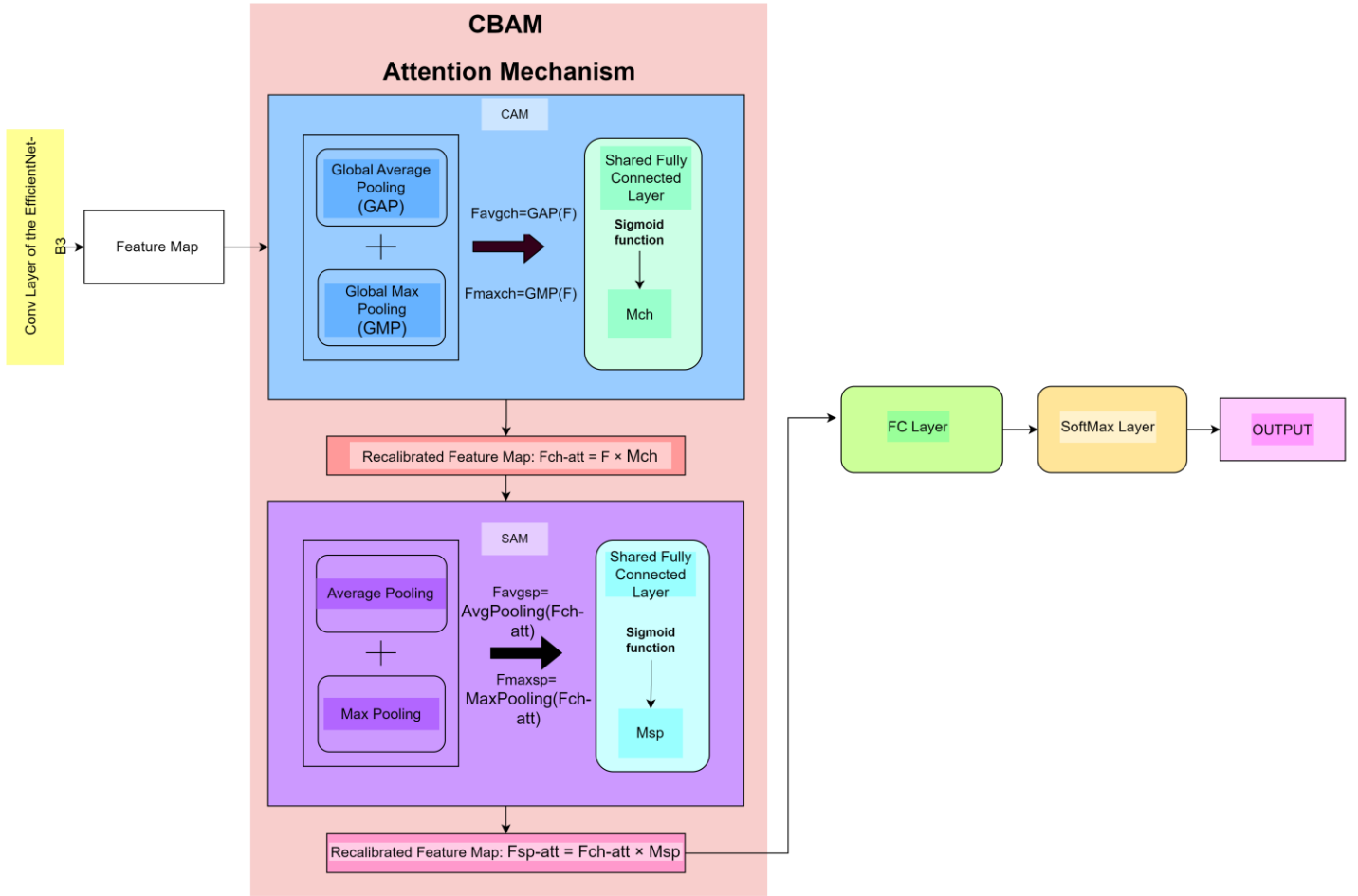


Figure 17: The detailed view of attention mechanisms enhancing feature representation. On the left side, input feature maps—representing intermediate representations from CNN layers—are processed by the Convolutional Block Attention Module (CBAM). The CBAM consists of two parallel pathways: Channel Attention (which operates along the channel dimension) and Spatial Attention (which works spatially across feature maps). These pathways involve operations like max pooling, average pooling, and shared multi-layer perceptron (MLP). The refined feature map resulting from both pathways undergoes additional average pooling and max pooling before being processed by a Fully Connected Layer (FC Layer). A sigmoid activation function produces an intermediate output, which is further processed by a SoftMax layer to generate the final system output. Attention mechanisms enhance learning efficiency by emphasizing important features, crucial for tasks like image recognition. By integrating these attention mechanisms, the model can more effectively capture subtle differences in spike waveforms, leading to improved classification accuracy and robustness. For spike image implementation Refer 2.6.2. Pretrained Model Selection

CBAM is implemented here to refine the feature maps generated by the convolutional layers of EfficientNet-B3 by applying sequential attention mechanisms. It operates through two main

components: Channel Attention Module (CAM) that focuses on the importance of each channel in the feature maps and Spatial Attention Module (SAM) which focuses on the importance of specific spatial locations within the feature maps. CAM pays attention to the meaningful content of the input image, compresses the input image in the spatial dimension, and aggregates the spatial information of the feature mapping using global average pooling and maximum pooling to generate channel attention; SAM pays attention to the location information of the input image, a compression of the input image in the channel dimension, and pooling in the channel dimension to form effective feature descriptors to generate spatial attention. Compared with the channel-only SE attention mechanism, CBAM attaches more significance to the location information of the image and can be flexibly embedded into the neural network to improve the feature representation capability without adding significant overhead. The construction of channel attention and spatial attention in CBAM.

Given an intermediate feature map $F \in RC \times H \times W$ as input, it is first processed by global average pooling and maximum pooling to obtain two different spatial feature descriptors F_{avg} and F_{max} , denoting average pooling features and maximum pooling features respectively, with size conversion to $RC \times 1 \times 1$. Then they pass through the shared FC layer module, in which the number of channels will be compressed to $1/r$ (reduction rate) times the original number of channels and then expanded to the original number of channels, and after the ReLU activation function the two are summed element by element to generate the channel attention $MC \in RC \times 1 \times 1$ information. The channel attention is multiplied by element with the input F to obtain the channel attention adaptive feature map $F1 \in RC \times H \times W$. As shown in equation (28) and equation (29).

$$\begin{aligned} MC(F) &= \sigma(\text{MLP}(\text{AvgPool}(F)) + \text{MLP}(\text{MaxPool}(F))) \\ &= \sigma(W1(W0(F_{avg})) + W1(W0(F_{max}))) \end{aligned} \quad 28$$

$$F1 = MC(F) \otimes F \quad 29$$

where σ is the sigmoid function, $W0$, $W1$ are the MLP module weights with shared rights, and $\otimes$ denotes the corresponding element multiplication. Then $F1$ is taken as input, global maximum pooling and average pooling are performed to obtain two vectors of size $R1 \times H \times W$. The vectors are spliced and subjected to a 7×7 convolution operation, and after the Sigmoid activation function, the final spatial attention $MS \in R1 \times H \times W$, as shown in equation (30) and equation (31).

$$MS(F) = \sigma(f_{7 \times 7}([\text{AvgPool}(F); \text{MaxPool}(F)])) = \sigma(f_{7 \times 7}[F_{avg}, F_{max}]) \quad 30$$

$$F2 = MS(F1) \otimes F1 \quad 31$$

where $f_{7 \times 7}$ is a 7×7 convolution operation $S; F_{max} S$). The range of the receptive field is determined by the size of the convolutional operation. A larger receptive field consolidates a broader range of contextual messages, making the representation of spatial attention more powerful. The use of 7×7 convolutional kernels in SAM to extract spatial features, compared to the use of small 3×3 convolutional kernels, increases the receptive field but also increases the number of parameters, which undoubtedly limits the application of CBAM [40].

EfficientNet-B3 and CBAM together provide a powerful framework for spike classification. EfficientNet-B3's architecture, with its compound scaling and MBConv blocks, efficiently extracts complex patterns from spike images. CBAM further enhances this capability by applying attention mechanisms that allow the model to focus on the most informative features, both channel-wise and spatially. This combination enables highly accurate classification of neuronal spikes, leveraging pretrained weights to minimize training requirements while maximizing performance.

Model Fine-tuning

For the ITSC model the steps involved in fine-tuning are as follows:

- a) *Unfreezing Layers:* After the initial training phase with frozen layers, the last few layers of the network are unfrozen. This selective unfreezing allows the model to adapt the high-level features to the specific characteristics of the spike waveforms.
- b) *Setting a Lower Learning Rate:* During fine-tuning, the learning rate is set to a lower value of 0.0001 to prevent drastic changes to the pretrained weights. A lower learning rate ensures that the adjustments made to the weights are subtle, preserving the generalization capabilities of the model while fine-tuning it to the new task.
- c) *Training with Backpropagation:* The model is trained using backpropagation, where the gradients are computed and used to update the weights of the unfrozen layers. The loss function, here cross-entropy loss for classification tasks, measures the difference between the predicted and actual class labels. The optimizer, Adam, updates the weights to minimize this loss.
- d) *Monitoring and Early Stopping:* During fine-tuning, the model's performance on a validation set is monitored to prevent overfitting. Early stopping is employed to halt training if the validation performance does not improve for a specified number of epochs (here 8 epochs). This technique ensures that the model maintains its ability to generalize to unseen data without overfitting.

Model Optimization

To achieve faster training, specific hardware and software configurations are recommended. Training deep neural networks, especially those with complex architectures like EfficientNet-B3, requires powerful GPUs (Graphics Processing Units) such as NVIDIA A100 or V100. These GPUs offer massive parallelism and high memory bandwidth, crucial for handling large datasets and deep models. For the project, a GPU with 80 Teraflops was used for faster training. The software configurations are as below:

a) Mixed Precision Training

GradScaler and Autocast: Mixed precision training involves using both 16-bit (half-precision) and 32-bit (single-precision) floating-point operations to accelerate training while maintaining

model accuracy. In PyTorch the *torch.cuda.amp.GradScaler* and *torch.cuda.amp.autocast* utilities were used to automatically scale gradients and cast operations to the appropriate precision.

$$Loss = CrossEntropy(f_{EfficientNet}(autocast(I)); \odot) \quad 32$$

The scaled loss is then computed as:

$$scaled_{loss} = GradScaler.scale(Loss) \quad 33$$

Mixed precision training reduces the memory footprint and increases computational throughput on GPUs, allowing for larger batch sizes and faster training. It also mitigates the risk of numerical instability by scaling gradients before backpropagation, preventing underflow in 16-bit calculations [41]. This approach is particularly beneficial for large-scale datasets, as it enables more efficient utilization of GPU resources.

b) *cuDNN.benchmark*

The *torch.backends.cudnn.benchmark* flag was set to True to allow the cuDNN library to optimize the selection of the fastest convolution algorithms based on the hardware and input size. The convolution algorithm selection can be represented as:

$$conv_algorithm = cuDNN.benchmark(input_{shape}, GPU) \quad 34$$

Enabling this feature allows cuDNN to dynamically choose the most efficient convolution algorithm during runtime, which can significantly speed up training on large-scale data by optimizing GPU utilization [42]. This dynamic selection process ensures that the most suitable algorithm is used for each specific hardware configuration and input size, enhancing overall training efficiency.

c) *Gradient Accumulation*

Batch Size and Gradient Accumulation: Due to memory constraints, the effective batch size was increased by accumulating gradients over multiple mini batches. Specifically, a batch size of 128 was used with gradient accumulation steps of 4, resulting in an effective batch size of 512. The accumulated gradient can be expressed as:

$$Accumulation_Gradient = \sum_{i=1}^4 \nabla_{\odot} Loss_i \quad 35$$

Gradient accumulation enables the training of large models on high-resolution images with limited GPU memory. By accumulating gradients over several forward passes before performing a backward pass, the model can simulate the effect of a larger batch size, which is crucial for stabilizing training in deep networks [43]. This technique allows for more stable and efficient training, particularly when dealing with large datasets and complex models.

c) Optimizer and Learning Rate

The Adam optimizer was used with an initial learning rate of 0.0001. The learning rate was fine-tuned using a learning rate scheduler that reduces the learning rate if the validation accuracy plateaus. The parameter update rule for Adam with gradient scaling can be represented as:

$$\theta = \theta - \alpha \cdot \text{GradScaler.unscale}(\text{Accumulated_Gradient}) \quad 36$$

The combination of gradient accumulation and the Adam optimizer allows for efficient handling of the large dataset and high-dimensional parameter space. The adaptive learning rate of Adam is particularly useful in fine-tuning, as it adjusts to the gradient variations in different layers of the network, enhancing convergence [44]. This adaptive nature of Adam helps in achieving faster convergence and better performance, especially in deep learning models with complex architectures.

The optimizations were crucial for enhancing the performance and efficiency of the model. Initially, the model's training process was significantly slower, with each epoch taking approximately 41 minutes to complete. This slow performance was primarily due to the large dataset size of ~1.4 million greyscale spike images. By implementing mixed precision training, cuDNN benchmarking, and gradient accumulation, the training time was substantially reduced. Mixed precision training reduced the memory footprint and increased computational throughput, allowing for larger batch sizes and faster training. This optimization alone contributed to a noticeable decrease in training time. Enabling cuDNN benchmarking allowed the model to dynamically select the most efficient convolution algorithms, further speeding up the training process. Gradient accumulation enabled the use of an effective batch size of 512, which stabilized the training and improved convergence rates. The above processes kept the memory (RAM) of the system at a constant 23% and the swap space at 69-70%, effectively using the system better (Refer Table 5: Overall Model Optimizations).

Model Development Code

All the models are implemented in python using PyTorch and run on Ubuntu system.

- a) Spike-based Classifier (SC) Model: github.com/Spike-based-Classifier-SC
- b) Image-based Transfer Learning Spike Classifier (ITSC) Model: github.com/Image-based-Transfer-Learning-Spike-Classifier-ITSC
- c) Image-based Spike Classifier Non-pretrained (ISC) Model: github.com/Image-based-Spike-Classifier-Non-pretrained-ISC

These repositories include the full code and necessary instructions for implementing and training each model.